COSMOS-ITS: Collaborative Orchestration System for Multi-Agent Optimization and Synthesis for Intelligent Tutoring Systems

By

Abdullah Al Masud Bhuiyan - 011221074
Kamrul Islam Arnob - 011221170
Sadik Mahmud - 011221113
Mahfuz Hasan Reza - 011221203
Kaium Al Limon - 011221291
Md Musfiqur Rahman - 011221334

Submitted in partial fulfilment of the requirements
of the degree of Bachelor of Science in Computer Science and Engineering

June 30, 2025



Department of Computer Science and Engineering
United International University

Abstract

COSMOS-ITS is personalized academic support for university students provided by an AI-powered intelligent tutoring system. With the traditional educational methods, it is not easy to develop scalable and adaptable learning material which fits in divergent university curricula. To address these limitations, COSMOS-ITS leverages a multi-agent architecture that incorporates state-of-the-art machine learning models, semantic vector search and domain-specific AI agents to provide context-aware re al-time tutoring. The system is organized as a layers from low-level text processing to end user interaction where it includes a guided chat interface, an supervised query classifier for domain routing, custom academic agents that include tools for content anyway retrieval and explanation and a contin uous AI feedback loop with adaptive learning. The development was modular and iterative, relying on tools such as FastAPI, ChromaDB or Manim. Evaluation shows COSMOS-ITS can successfully identify and classify queries, ensure relevant multi-modal responses and personalise help based on user history both known (profile) and unknown (feedback). Conclusion: This innovation provides an adaptive and personalized platform to simulate individual academic guiding in school and then improve learning experiences of students as well as increase success in the classroom.

Acknowledgements

Credit for the successful accomplishment of this project is due to the co-operation and assistance of numerous individuals. We wish to thank everyone that helped the project to advance.

We are very thankful to the honorable supervisor **Dr. Hasan Sarwar** for his precious directed, constructive criticism and endless support during the project.

In addition, we would like to thank our classmates for their kind support and constructive feedback that inspired us to continue when times became tough.

Finally our families, and especially our parents are deeply thanked for there endless emotional support and unconditional love.

Table of Contents

Table of Contents iv

List of Figures v

List of Tables vii

1 Introduction 1
- 1.1 Project Overview . . . 1
- 1.2 Motivation . . . 2
- 1.3 Objectives . . . 3
- 1.4 Methodology . . . 4
  - 1.4.1 UI Layer – Student Interaction Portal . . . 4
  - 1.4.2 Orchestration Layer – Query Routing & Task Assignment . . . 5
  - 1.4.3 Agent Infrastructure Layer – Specialized Academic Agents . . . 5
  - 1.4.4 Database Layer – Persistent and Vector Storage . . . 6
  - 1.4.5 AI Feedback Loop: Learning from Responses . . . 7
  - 1.4.6 Development Phases and Tools . . . 7
  - 1.4.7 Summary of Methodological Strengths . . . 8
- 1.5 Project Outcome . . . 8
- 1.6 Organization of the Report . . . 8

2 Background 10
- 2.1 Preliminaries . . . 10
- 2.2 Literature Review . . . 11
  - 2.2.1 Similar Applications . . . 11
  - 2.2.2 Related Research . . . 12
- 2.3 Gap Analysis . . . 13
  - 2.3.1 Identified System Gaps . . . 13
  - 2.3.2 SWOT Analysis . . . 14
- 2.4 Summary . . . 15

3 Project Design 16
- 3.1 Requirement Analysis . . . 16

3.1.1 Functional and Nonfunctional Requirements . . . 17
3.1.2 Context Diagram . . . 22
3.1.3 Data Flow Diagram Level 1 . . . 23
3.1.4 UI Design . . . 24
3.2 Detailed Methodology and Design . . . 32
3.2.1 Methodology: Multi-Agent LLM Architecture . . . 32
3.3 Project Plan . . . 35
3.4 Task Allocation . . . 37
3.5 Summary . . . 41

4 Implementation and Results 42
4.1 Environment Setup . . . 42
4.2 Testing and Evaluation . . . 47
4.3 Results and Discussion . . . 49
4.4 Summary . . . 51

5 Standards and Design Constraints 52
5.1 Compliance with the Standards . . . 52
5.1.1 Software Standards . . . 52
5.1.2 Hardware Standards . . . 54
5.1.3 Communication Standards . . . 55
5.2 Design Constraints . . . 56
5.2.1 Economic Constraint . . . 56
5.2.2 Ethical Constraint . . . 56
5.2.3 Social Constraint . . . 56
5.2.4 Political Constraint . . . 57
5.2.5 Sustainability . . . 57
5.3 Cost Analysis . . . 57
5.3.1 Development Cost . . . 58
5.4 Complex Engineering Problem . . . 58
5.4.1 Complex Problem Solving . . . 58
5.4.2 Engineering Activities . . . 61
5.5 Summary . . . 63

6 Conclusion 64
6.1 Summary . . . 64
6.2 Limitation . . . 65
6.3 Future Work . . . 66

References 67

List of Figures

2.1 Feature comparison of COSMOS-ITS with existing AI tutoring platforms . 13

3.1 Context Diagram 22
3.2 Level 1 DFD: Internal Module Interaction within COSMOS-ITS 23
3.3 High-Level System Architecture: API Gateway and Multi-Agent Flow . . . 23
3.4 Landing Page 24
3.5 Chatbot UI 24
3.6 performance Tracking UI 25
3.7 Roadmap UI 25
3.8 Quiz UI 26
3.9 Study Planner UI 26
3.10 Dashboard 27
3.11 Add Question 27
3.12 Uploaded Question 28
3.13 Course wise question 28
3.14 Server Content 29
3.15 Course Management 29
3.16 New Agent 30
3.17 Help and Support 30
3.18 User Manage 31
3.19 System Pipeline of COSMOS-ITS 32
3.20 Project Plan 35
3.21 Project Ideation 37
3.22 Requirement Analysis 37
3.23 Tech Stack and Environment Setup 37
3.24 AI Model Research and Analysis 38
3.25 Backend AI Design with Minimal Frontend 38
3.26 Optimized Backend API 38
3.27 Tokenization and Performance Improvement 39
3.28 Workflow Research and UI 39
3.29 Complete Backend and Documentation 39
3.30 Initial System Implementation 40

3.31 Report and Poster for FYDP-1 40

4.1 Side-by-Side Admin and Cliant Repository Folder Structures 46
4.2 Load test configuration and realtime stats via Locust 48
4.3 Response time and request throughput via Load Test 50

5.1 Cost Analysis 57

List of Tables

3.1 Summary Table of Design Decisions of COSMOS-ITS 34
3.2 Table of Project Planning of COSMOS-ITS 36

5.1 Development Team Cost Breakdown 58
5.2 Mapping with complex problem solving. 59
5.3 Mapping with complex engineering activities. 61

Chapter 1

Introduction

1.1 Project Overview

In the past few years, the need of customized learning and scalable academic support systems has exposed traditional university level-classes for what they are: not enough. Learners commonly experience embarrass or even humiliation when they can't obtain access to real-time personalized support because of a shortage of faculty members, high student-faculty ratios and non-adaptive class room approaches. There are academic counseling and online resources, but they often aren't contextually tailored to a university's curriculum and do not change in response to shifts in a student's learning profile.

Today's education systems are beginning to see artificial intelligence as a powerful tool to bring together standardized teaching and individual learning. However, these platforms generally do not provide context-sensitive adaptive and continuous support suited to the specific academic organisation in different universities nor to accommodate the varying students' needs.

This project is geared toward turning static educational interactions into dynamic, customized learning experiences through the use of a collaborative system of intelligent agents that are specialized for various tasks including content recovery, code interpretation, visualization and student modeling. The platform is designed to serve as an ever-present, smart tutor able to mimic that zeal and adaptability of a human teacher - on mass. It uses a multi-agent design with sophisticated models for student modeling and reflection, domain delivery, and context based tutoring.

Background:

College students are burdened with significant academic pressure and don't always have consistency nor the personalized support they need. Even using traditional classroom delivery it is often difficult to ensure that resources are matched to the way students learn, and off-the-peg teaching materials do not necessarily allow for the format and emphasis of individual institutional programmes of study. Furthermore, academic counseling services

are limited by faculty time and scheduling constraints that make it difficult to intervene with one-on-one contacts with students who need help.

Problem Statement:

No intelligent tutoring systems that can provide on-line, university specific, tailored support which is responsive to the students in real-time and as detailed as an individual may need have yet been developed. Current learning platforms do not incorporate syllabus mapped content, customized analytics and round the clock support on a single platform. This gap will be covered by the proposed project through the development of an AI grounded in education framework that is capable to deliver persistent, context- and expert-informed academic advising for every student in relation to her/his bench-marking institution factors and learning history.

1.2 Motivation

Nowhere is this more true than in higher education, where the struggle to make sure each and every student has the academic support they need rages on. Even though the Internet and digital content are ubiquitous, one-size-fits-all models of learning still predominate in most education systems. These approaches are too restricted to meet the own learning pace and difficulties, priorities of the individual students concern. This critical gap, which so greatly affects learning outcomes, is created by the difference between student resources and personalized guidance.

Conventional classroom method, especially in the university setups, has a lot of students at once and thus the teachers are often unable to give attention to each student personally. Although academic counseling and office hours serve to meet specific needs of students, these modality is limited by time and specificity, inconstant when needed, it not available for all student at their most critical period. Also online learning like videos, forums or lecture notes are great yet it is very passive and does not have the capability to figure out as well as react individually toward rawnesses. Most of the time students are left to their own devices to recognize their mistakes without any solid direction, resulting in errors and frustration then eventually demoralized disconnection.

Such void widen further, when we there are technical and conceptual subjects as in Software Engineering, Database Management Systems or Algorithms where clarity of concept are practice are both necessary. Here is where timely and context-aware feedback can make all the difference between mastery or confusion. But people don't scale; human teachers aren't realistically able to provide the 24/7 support that kids need to learn effectively, or for each student to address his or her unique cognitive profile.

The rationale for **COSMOS-ITS (Collaborative Orchestration System for Multi-Agent Optimization and Synthesis For Intelligent Tutoring Systems)**, is to ad-

dress the pressing need on this point. The aim is nothing short of to develop a system that has the attentiveness of a personal tutor along with the scale and intelligence of contemporary AI. Through the use of multi-agent collaboration, contextual understanding of questions asked and personalized response generation, COSMOS-ITS attempts to emulate the advantages provided by one-on-one tutoring, but in this case removing the availability constraints posed by humans so that all students have access to help when they need it.

It should not be just a question-answering system, but the system also needs to understand, learn and optimize answers with respect to learner's behavior history, user current goal and course context information. It is able to classify the course domain, to identify the appropriate agent and find accurate knowledge from user-learnt databases answering with tailored communication strategies. That brand of automation and customization could revolutionize how students interact with educational material and address learning differences.

If the conclude, the inspiration for COSMOS-ITS is driven by a real-life high-stakes educational problem. It aims to make students feel they have the power by:

- Reducing the reliance on faculty time and manual tutoring, which are limited resources.
- Offering constant smart academic support.
- Dynamically addressing student's weaknesses and interests.
- Improving the quality and entertainment level of self-learning.

This project represents a stride toward turning AI-powered personalized tutoring into a reality—closing the gap between information we can access and education that truly works.

1.3 Objectives

COSMOS-ITS's main goal consists in transforming university education by providing hyper-personalized AI-based tutoring that evolves with the learning process, the context of each student and particular university curriculum at real time. This is realised through the support for smart orchestration, situation awareness and multi-agent cooperation in a modular and scalable setup.

1. **Simulated AI Tutor:**

   To generate a personal academic tutor through the use of AI agents, which can understand student intention, historical weaknesses and immediate requirement.

2. **Orchestration System Design:**

   Create an orchestration system for rerouting a query dynamically to appropriate domain-specific agents (e.g., SE-Agent, DBMS-Agent) or fallback strategies considering its query complexity and course classification.

3. **Contextual Retrieval with RAG:**

   To incorporate smart retrieval mechanisms (RAG) using vector databases (like ChromaDB) and banks of questions for robust, contextually-relevant responses.

4. **Multimodal Explanation Support:**

   To facilitate interactive, multimodal explanations methods like code synthesis, diagram rendering with Manim, and document parsing (with Docling) to deepen understanding of concepts.

5. **Continuous AI Feedback Loop:**

   The solution is a continuous AI feedback loop to assess quality of agent responses and change tutoring strategies dynamically – a system that gets better as it's used.

6. **Memory-Aware Learning:**

   To enable memory-aware learning by capturing and recalling session style and content, historical interaction, and changing user preference in order to preserve long-term continuity of tutoring.

7. **Scalability and Modularity:** There should be a possibility to extend the system in order make it available for new courses, agents, universities.

1.4 Methodology

The methodological approach employed for the construction of COSMOS-ITS (Collaborative Orchestration System for Multi-Agent objective optimization and Synthesis for Intelligent Tutoring Systems) is based on a layered, modular and agent-based methodology which focuses on developing an efficient, intelligent and personalized tutoring system suitable for university level students. The latter is configured to behave as a real-time interactive assistant that can react to various academic questions in several subjects thanks to context-aware AI agents, coordinated by a supervised routing layer.

The entire methodology is divided into the following core layers as per the finalized architecture:

1.4.1 UI Layer – Student Interaction Portal

This layer serves as the primary interface between the user and the system. It is a web-based chat UI, built with modern frontend technologies and connected to a FastAPI server.

Features:

- Chat-based interface for natural language queries.
- Context preservation through session tracking.
- Real-time communication using WebSockets or HTTP.

Purpose:

- To provide students with a simple, intuitive, and responsive platform for academic interaction.
- To pass user queries, history, and metadata to the backend orchestration engine.

1.4.2 Orchestration Layer – Query Routing & Task Assignment

At the heart of the system is the Orchestration Layer, which acts as the intelligent conductor of agent tasks.

Components:

- Course Classifier (Supervised Model): Classifies incoming queries into academic domains such as Software Engineering, DBMS, Mathematics, etc.
- Intent Recognizer: Determines if the query is informational, conceptual, example-based, or practice-driven.
- Routing Engine: Based on classification and intent, routes the task to the appropriate agent or fallback strategy.

Logic Flow:

- Accept the query and context from the UI Layer.
- Identify course domain using a trained ML classifier (e.g., Scikit-learn or custom transformer).
- Identify task type (e.g., "Explain," "Solve," "Visualize").
- Route the request to the best-matching agent via the Agent Infrastructure Layer.

1.4.3 Agent Infrastructure Layer – Specialized Academic Agents

This layer contains the domain specific AI agents that actually do the academic reasoning and content generation. Each agent functions in an semi-autonomus manner with specific tools available.

Types of Agents:

In COSMOS-ITS, agents are modular and can be highly customized; this permits different types of agents to be designed according to several parameters such as subject domain, query intent, difficulty level, or learning goals. For instance, agents for specific courses

such as SE-Agent and DBMS-Agent only train on course material but other agents might be specialized for specific tasks, e.g., quiz generation, code explanation or summarization. AGENTS can also vary by behavior that means some are detailed on the day-to-day steps needed to execute a plan, others focus primarily on providing high-level conceptual clarity . This flexible model of agency supports tuning the COSMOS-ITS environment to integrate with a wide variety of academic requirements for new courses, varying curricula, and differences in student learning styles.

Tools Available to Agents:

- Vector Similarity Search: Querieees ChromaDB to retrieve most relevant content (e.g., lecture notes, slides).
- Q-Bank Search: Finds curated Qpairs for structured problem solving.
- Docling Integration: It reloads the file and parses outs info for academic documents.
- Manim Engine: Used to produce dynamic math/code visualizations for better understanding.
- Code Interpreter: Executes and interprets code snippets as required.

Agent Task Execution Steps:

- Get a job from the Orchestrationn Layer.
- lookup with ChromaDB and/or Q-Bank based on context.
- Run pipelines specific to tools if necessary like, output a diagram with Manim.
- Format your answer and return it (together with an explanation).

1.4.4 Database Layer – Persistent and Vector Storage

This layer provides a flexible, highly scalable and context aware processing of the data, organized in two independent subsytems:

Session Store (Redis or similar):

- Stores per-user chat history, preferences, learning behaviour.
- Guarantees memory conscious tutoring (such as "You had problems now with ER Diagrams...")

Vector Database (ChromaDB):

- The of Records embeddings of lecture notes, slides, past Q&A, definitions store.
- Enables search based on high level query matching heree.
- Updated with fresh content regularlies, by ingestion pipelines.

1.4.5 AI Feedback Loop: Learning from Responses

One of the important methodological approaches in COSMOS-ITS is its AI feedback mechanism for closing the loop between action and evaluation.Is shown below:.

How It Works:

- At the conclusion of each response, two such test for:
  - Success in resolving queries (Did it require further action?)
  - Student Evaluation/Response "Helpful", "Needs clarification")
  - Accuracy of the response and quality of output 2.

Adaptation Logic:

- If this is non-optimal response received then:
  - Re-routing the logic is tuned for similar future queries.
  - The agent knowledge base is promotable for enrichment.
- If successful:
  - The response type is stored as a default template for other intents that function the same way.

1.4.6 Development Phases and Tools

The implementation follows an iterative, incremental development model, incorporating both rule-based and learning-based AI techniques.

Phase 1: UI & API Prototyping using FastAPI + HTML/CSS/JS.
Phase 2: Course classifier model development and validation.
Phase 3: Agent tool integration and ChromaDB ingestion.
Phase 4: Multi-agent orchestration and task routing.
Phase 5: Feedback loop integration and personalization logic.
Phase 6: Evaluation and testing with real university queries.

Key Technologies:

- FastAPI for backend orchestration
- Supabase + Redis for session & user data
- ChromaDB for semantic vector search
- Gemini API for LLM fallback support
- Crew AI for coordinating agent logic
- Manim, Docling, and Python for academic utilities

1.4.7 Summary of Methodological Strengths

- Reranking responses in the context of vector search and chat history.
- Modular agent architecture, extensible to cover other topics.
- An intelligent timeline is set to balance accuracy with efficiency.
- Developing a memory and feedback based personalized learning path.
- Extensible beyond Future AI Integration like adaptive quizzes, curriculum recommendations.

1.5 Project Outcome

The COSMOS-ITS service, supplied to university students, will be a functioning AI-based intelligent tutoring system that can offer immediate personalized support in educational needs. The system will be able to:

- Properly categorize student questions and route them to appropriate domain-specific AI agents.
- Extract and combine context-specific responses with domain and parameter fine-tuned, trained knowledge bases.
- Personalize tutoring according to the past learning histories, query patterns and feedback.
- Can simulate human-like academic guidance, which is always available, scaling well and domain-aware.

Students will, at the end of the project, engage with COSMOS-ITS as their bespoke digital tutor – one that knows where they are struggling, can provide tailored assistance and grows with them in their learning journey.

1.6 Organization of the Report

The report is divided into six main chapters, each of which concentrates on a different aspect of the project. The organization is as follows:

- **Chapter 1: Introduction**

  In this chapter, we give an introduction to the project: Problem Statement, Motivation, Objectives, and a brief discussion of methods used.

- **Chapter 2: Background**

  The chapter discusses the basic concepts, research work and literature support for the present work.

- **Chapter 3: Project Design**

  System Design This chapter describes the implementation of the proposed solution on a high level and elaborates its design which consists of system architecture, data flow and component interfaces.

- **Chapter 4: Implementation and Result**

  This chapter explains how the system was built, the technologies applied, as well as discusses the results and performance obtained by testing.

- **Standard and Design Constraints**

  The standards adhered and limitations taken into account in the design and implementation process of this project will be covered in this chapter.

- **Chapter 6: Conclusion**

  The last section concludes this thesis, describes future work and presents discussion of the findings.

Chapter 2

Background

2.1 Preliminaries

In the context of studying AI-based educational platforms, several related systems have been identified. However, many of them focus on partial features, aim at other goals, or are not flexible and personalizable. Only a few systems include adaptive feedback, performance monitoring, personalized learning resources, and intelligent multi-agent interaction all operating together in a single environment.

Unlike **COSMOS-ITS** (Collaborative Orchestration System for Multi-Agent Optimization and Synthesis for Intelligent Tutoring Systems), our project addresses these limitations by providing a full suite of AI-engineered capabilities tailored for higher education. COSMOS enables self-regulated learning, allowing students to monitor progress, receive customized feedback, and follow automatically generated study plans based on performance.

One of the main parts of COSMOS-ITS is its Multi-Agent System (MAS), in which a set of agents specializes on university courses or activities, like classification, answering questions and study plan generation. Coordination between these agents happens via an *Orchestration Layer* that employ intent classification and path detection to route the request to right agent. This guarantees that each request is handled contextually by the right AI service.

For context-aware replies, COSMOS uses LLM-based Retrieval-Augmented Generation (RAG). Education like slides, notes and Qbanks are saved as vectors in vector databases such as `ChromaDB` or `Pinecone`. These are converted into embeddings that retain the semantic information such that vector similarity search work is used to retrieve items with the most similar content.

[1] A main benefit of COSMOS is the use of multimodal input, where users can com-

[1]The system also incorporates `Docling`, a document.grounded LLM toolkit for agents to answer ques-

municate through means such as text, code, diagrams, and mathematical expressions to dialogue with models like `Gemini`. Conversations are classified using intention detection based on NLP which infers whether a student wants doubt-solving, quiz, or learning path updates.

It predicts weaknesses from manualeno inputs (e.g., exam scores) and system interactions to funded personalized learning paths. The Q-Bank Search feature lets students search for questions from the past and topic-specific query through semantic search, not just keyword matching.

In this way, COSMOS-ITS is a novel entity in educational platforms space because it is an end-to-end intelligent adaptive environment where pedagogy and AI technologies design are fully integrated.

2.2 Literature Review

This chapter reviews initial work and recent literature. A total of 28 papers have been analyzed: 16 academic papers and 12 AI-based educational applications. Among them, 9 representative papers and 9 related applications were selected based on methodological significance, computational complexity, and comparability. This review highlights the advantages and disadvantages of existing Intelligent Tutoring Systems (ITS), AI-based learning support, and performance tracking, which form the basis for the design of COSMOS-ITS.

2.2.1 Similar Applications

Several AI-based educational platforms have been developed to support adaptive content delivery and intelligent tutoring. Most platforms specialize in a single aspect of learning rather than providing full-featured ecosystems. Here, four relevant platforms are discussed in comparison to COSMOS-ITS.

Khanmigo

Khan Academy's GPT-4-powered tutor uses a Socratic guided-learning approach, asking probing questions instead of providing direct answers. It is closely integrated with Khan Academy content (math, science, humanities, coding) and provides real-time hints, scaffolding, and feedback. Instructors can track lesson plans, formative assessments, chat histories, and student progress. Safety features include monitored chats. [1]

tions from course materials. A multi-robot coordination framework `Crew AI` is also employed to coordinate the interactions between the course agents,lanners and evaluators.

QANDA (Mathpresso)

QANDA is an AI-based online math solver. By early 2024, it analyzed 6.3 billion questions from 90 million users. MathGPT, developed in-house, enhances math reasoning significantly. Features include solution summaries, community Q&A discussions, flashcard-based study tools ("Cramify"), a custom quiz builder, and progress tracking. [2]

LearningRO

Founded in 2025 in the UK, LearningRO provides the RoTutor AI chatbot for GCSE, BTEC, and A-Level exams. It delivers step-by-step instruction, personalized flashcards, and quizzes. The Performance-based adjustments guide students along tailored study paths. The platform supports multimodal contents like tactile, visual, multiple-choice and so on. [3]

Kira Learning

Created by Andrew Ng, Kira Learning uses AI agents for tutoring and teacher support. The suite includes:

- **AI Tutor:** Real-time advice and personalized guidance.
- **AI Teaching Assistant:** Monitors students and assists teachers.
- **AI Grader:** Automation of assignment feedback and analytics.

The system supports multimodal input and integrates with LMS platforms. [4]

Benchmark Analysis: Figure 2.1 compares COSMOS-ITS with existing AI tutoring platforms in terms of topic-wise AI agents, weakness prediction, study plans, personalized learning paths, performance tracking, quiz support, and session history. Most platforms support quizzes and limited AI tutoring, but none offer continuous weakness tracking or dynamic personalized memory-based feedback like COSMOS-ITS.

2.2.2 Related Research

Recent studies demonstrate the application of AI and LLMs in education for personalized learning, early intervention, and intelligent tutoring:

- Khan et al. [5] used decision trees to predict underperforming students with 86% accuracy.
- Córdova-Esparza et al. [6] conducted a literature review of 82 AI tutoring systems, highlighting best practices and ethical concerns.
- Bassner et al. [7] assessed a virtual tutor "Iris" with RAG, showing improved engagement and understanding.

Platform	Topic-wise AI Agents	Weakness Prediction	Study Plan (To-Do List)	Learning Path	Performance Tracking	Interactive Quizzes	Session History / Review
COSMOS-ITS	✔	✔	✔	✔	✔	✔	✘
VTutor	✔ (limited)	✘	✘	✘	✘	✔	✘
Khanmigo (Khan Academy)	✔	✘	✘	✘	✘	✔	✔
Alice.Tech	✔ (tech-focused)	✘	✘	✘	✘	✘	✘
Kira Learning	✘	✘	✔ (static/basic)	✘	✘	✔	✘
Brainly AI Tutor	✔	✘	✘	✘	✘	✘	✔
VEGA AI	✘	✘	✘	✘	✘	✘	✘
Synap	✘	✘	✔	✘	✘	✔	✘
QANDA (Mathpresso)	✘	✘	✘	✘	✘	✔	✘
LearningRO	✔	✘	✘	✘	✘	✘	✘

Figure 2.1: Feature comparison of COSMOS-ITS with existing AI tutoring platforms

- Liu et al. [8] applied GPT-4V-based multimodal ITSs for language learning using a scaffolding approach.
- Vaswani et al. [9] introduced the transformer model, enabling parallel computation and becoming the foundation for GPT.
- Yeung et al. [10] demonstrated zero-shot LLMs for automated grading and response.
- Livathinos et al. [11] presented Docling for fast document conversion via layout-aware OCR and transformers.
- Neumann et al. [12] developed MoodleBot, an LLM chatbot achieving 88% accuracy and supporting self-regulated learning.

These studies demonstrate the revolutionary potential of AI and LLMs in the field of education.

2.3 Gap Analysis

Although platforms like Khanmigo, VTutor, and Kira Learning exist, they do not provide system-wide or individualized personalization and adaptivity. COSMOS-ITS addresses these limitations.

2.3.1 Identified System Gaps

- **No Topic-wise Modular Agents:** Existing platforms lack modular agents for sub-topics.

- **Lack of Performance-Based Feedback:** Quiz/test performance is not efficiently used to guide content delivery.
- **No Dynamic Study Plans:** Current platforms rely on fixed study plans.
- **No Query Memory for Personalization:** Systems lack memory of student misconceptions.
- **Manual Academic Tracking Not Available:** Students cannot track performance across courses.
- **Poor Visualization & Gamification:** Limited dashboards, badges, or visual feedback.
- **Scalability and LMS Integration:** Minimal support for institutional scaling or LMS integration.

2.3.2 SWOT Analysis

Strengths

- Topic-specific AI agents for focused actions.
- Weakness prediction with targeted suggestions.
- Automated personalized study plans (to-do lists).
- Learning paths tailored to student performance.
- Memory-based suggestion engine for query history.
- Modular and scalable architecture.

Weaknesses

- Manual entry dependency for exam marks.
- Lack of real-time chat support.
- Onboarding curve for effective use of AI agents.
- Limited rich visual feedback and gamification.

Opportunities

- LMS and university integration (Moodle, Blackboard).
- Third-party service linking for suggestions.
- Gamification and progress visualization.

- Weekly summaries and targeted revision via memory-based feedback.
- Generative content agents for quizzes and flashcards.
- AI research and R&D integration.

Threats

- Intensifying competition from larger platforms.
- Compliance with privacy laws (e.g., GDPR).
- Resource demands from memory and vector search.
- User resistance due to AI decision-making.

2.4 Summary

COSMOS-ITS addresses gaps in AI tutoring systems with memory-based suggestion routing, topic-level modular AI agents, manual performance tracking, and dynamic study planning. The SWOT analysis highlights strengths and opportunities that support scalable, value-driven innovation.

Chapter 3

Project Design

3.1 Requirement Analysis

The design of COSMOS-ITS is based on a systematic study of the educational problem to be addressed, as well as the system it intends to build. As mentioned in the problem description, university students often struggle with complex course content because of lack of attention and custom help, time-limited faculty members, and ineffective static online resources. To fill this gap, the COSMOS-ITS project develops an AI-based multi-agent intelligent tutoring system able to supply with real-time, context-sensitive and adaptive academic assistance.

The architecture of the system is based on a multi-layered design (as described in Section 1.4), incorporating student engagement, course profiling, agent-based orchestration, knowledge retrieval and intelligent feedback loops. This architecture was derived following the consideration of end-users (students) learning behaviours, faculty constraints over resources and the automatically scalable intelligent support system.

This requirements analysis summarizes the key functionality of the system and roles of main users. These needs are further categorized into functional and non-functional in section 3.1.1. The context diagram gives a visual representation of the system and its environment such as interactions. Section 3.2.1 shows the context diagram for this project. Section 3.2.2 will describe the functional links between modules and Sections 3.2.3–3.2.4 cover the entity-relationship model and user interface design accordingly.

3.1.1 Functional and Nonfunctional Requirements

Functional Requirements

1. User Interface and Experience

- The application must provide a clean, conversational chat interface where students can interact naturally with topic-wise agents.
- The UI must support rich content rendering: text, images, code blocks, animations (e.g., via Manim), and clickable study resources.
- A responsive, mobile-friendly design must be implemented to support access across various devices.
- UI should include tabs or views for Chat, Study Plan, Quizzes, Performance Dashboard, and Feedback.

2. User Authentication and Profile Management

- Secure registration and login functionality must be provided.
- User profiles store:
  - Academic details: department, enrolled courses, batch/year.
  - CT and exam marks (manually input by students).
  - Learning preferences and history.
- Support for password recovery, multi-factor authentication, and role-based access (students, admins).

3. Multi-Agent Query Processing and Routing

- Implement a supervised classifier that automatically detects the subject/domain of a student's question.
- Route queries to domain-specific agents (e.g., SE-Agent, DBMS-Agent).
- Fallback agents must handle ambiguous/general queries.
- Dynamically switch agents mid-conversation if topic context changes.

4. Context and History Management

- Maintain complete chat history per user, enabling:
  - Continuity of multi-turn conversations.
  - Contextual reference in follow-up queries.
- Allow users to revisit previous responses.

5. Knowledge Base Integration and Vector Search

- Integrate with ChromaDB (or similar) to store and retrieve lecture slides, notes, Q&A banks.

- Implement semantic vector similarity search to pull relevant excerpts during student interactions.

6. AI Response Generation and Personalization

- Agents must generate answers based on student level (1st year vs. 4th year), current topic, and past questions.
- Support multi-modal responses using Docling, Manim, Code Interpreter, and visual explanations.

7. Weakness Prediction & Group Suggestion

- Based on quiz results and interactions, the system should:
  - Identify weak topics per student.
  - Suggest peer learning groups or study buddies based on common weaknesses.
  - Adapt the personalized learning path accordingly.

8. Study Plan (Smart To-Do List)

- Students can generate weekly or course-wise study plans.
- Each item should be trackable (marked as complete/incomplete).
- Integration with weakness prediction to auto-prioritize tasks.

9. Customised Learning Path

- Recommend a tailored sequence of topics based on performance, quiz outcomes, and interaction history.
- Update dynamically based on progress and new performance data.

10. Course Performance Tracker

- Students must be able to manually input CT marks and final exam marks per course.
- Data should be visualized in dashboards and used in weakness analysis and learning path adjustment.

11. Quiz and Assessment Module

- Topic-wise quizzes with MCQs or short answers.
- Show instant results, correct answers, and explanations.
- Maintain a quiz history and performance trend graph.
- Results inform weakness detection and study path updates.

12. Admin Dashboard and Monitoring

- Admins can monitor system metrics (agent accuracy, latency, usage), manage knowledge base content uploads, view/moderate flagged conversations, and oversee group suggestion system and performance analytics.

13. Feedback and Reporting

- Students must be able to rate agent responses and report incorrect answers.
- Feedback should be stored and used to refine agent behavior.

14. Multi-Platform Accessibility

- Platform should be accessible from desktop web browsers and mobile browsers/apps.
- Provide optional integration with university portals or LMSs (e.g., Moodle).

Nonfunctional Requirements

1. Performance

- Responses must be generated within 1–2 seconds to maintain a real-time experience.
- Capable of handling hundreds of concurrent users without delay.
- Quiz evaluation and vector searches should be optimized for low-latency retrieval.

2. Scalability

- Backend must scale horizontally to support new students and new course-specific agents/tools.
- Support sharded vector databases and distributed agent orchestration.

3. Security

- Data encrypted at rest and in transit.
- Implement OAuth/SSO support for university login integration.
- Protect all APIs with rate limiting, CSRF tokens, and secure token storage.
- Role-based access control ensures separation between student and admin functionalities.

4. Privacy and Compliance

- Comply with university data policies and global privacy laws like GDPR.
- All user data deletable upon request.
- Personal learning histories and scores visible only to respective students.

5. Reliability and Availability

- Target 99.9% uptime with auto-recovery and failover support.
- Include status monitoring dashboard and error logging.

6. Maintainability and Extensibility

- Modular microservices architecture allows easy addition of new course agents and updates to orchestration logic.
- here followed well-documented Codebase ,by following CI/CD practices.

7. Usability and Accessibility

- Comply with WCAG 2.1 standards: color contrast, keyboard navigation, screen reader support.
- Include multilingual support or easy localization options.

8. Interoperability

- Provide APIs for LMS, university databases, and third-party AI tools.
- Use REST or GraphQL with OpenAPI specs for documentation.

9. Auditability and Logging

- Every student interaction and system decision must be logged.
- Logs stored securely and accessible only by authorized admins.
- Support automated reporting for usage stats and academic insights.

3.1.2 Context Diagram

The context level diagram presents an overview of the system, showing only the main external entities interacting with it and how they communicate/interact (what data flows in/out of the system). It defines the scope of the system and identifies key players: students, teachers, and admin.

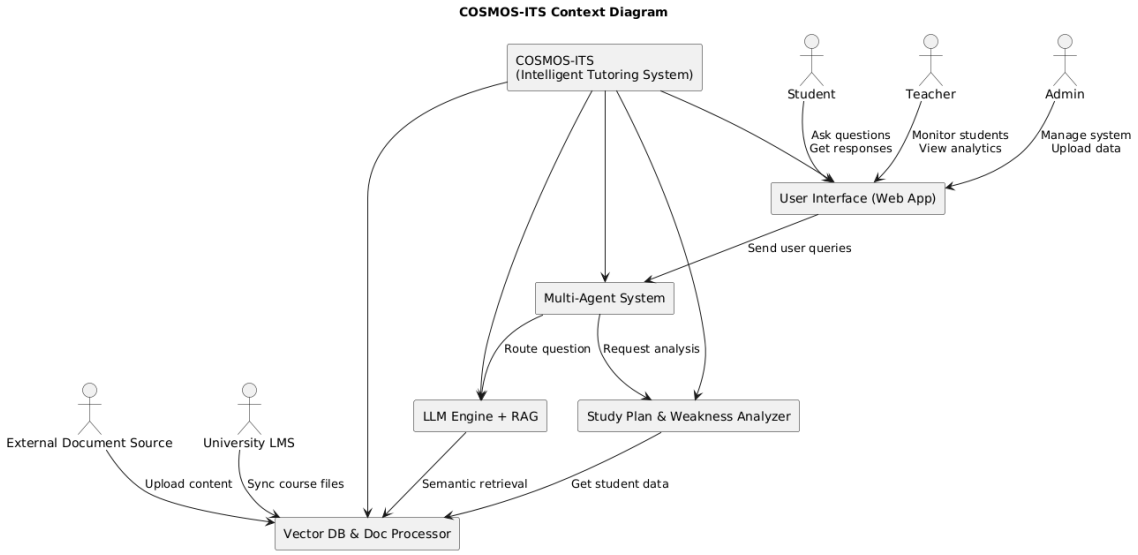


Figure 3.1: Context Diagram

3.1.3 Data Flow Diagram Level 1

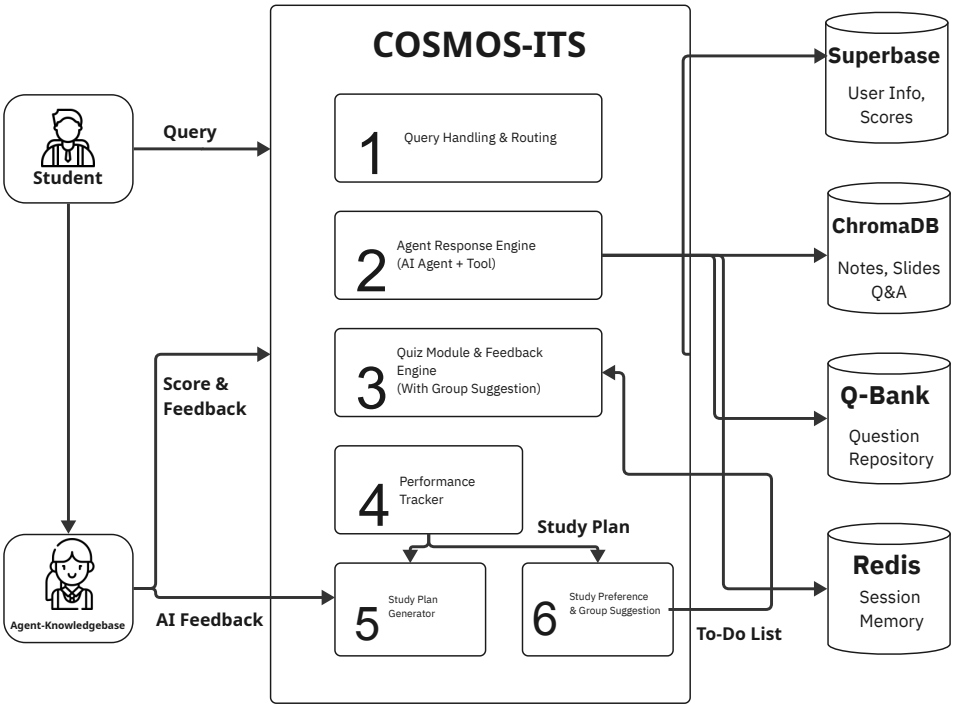


Figure 3.2: Level 1 DFD: Internal Module Interaction within COSMOS-ITS

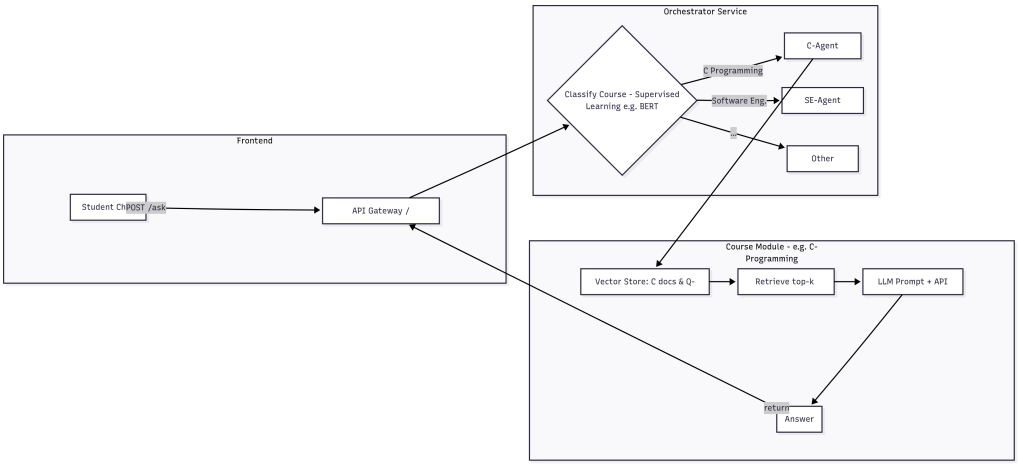


Figure 3.3: High-Level System Architecture: API Gateway and Multi-Agent Flow

3.1.4 UI Design

UI Design for Client

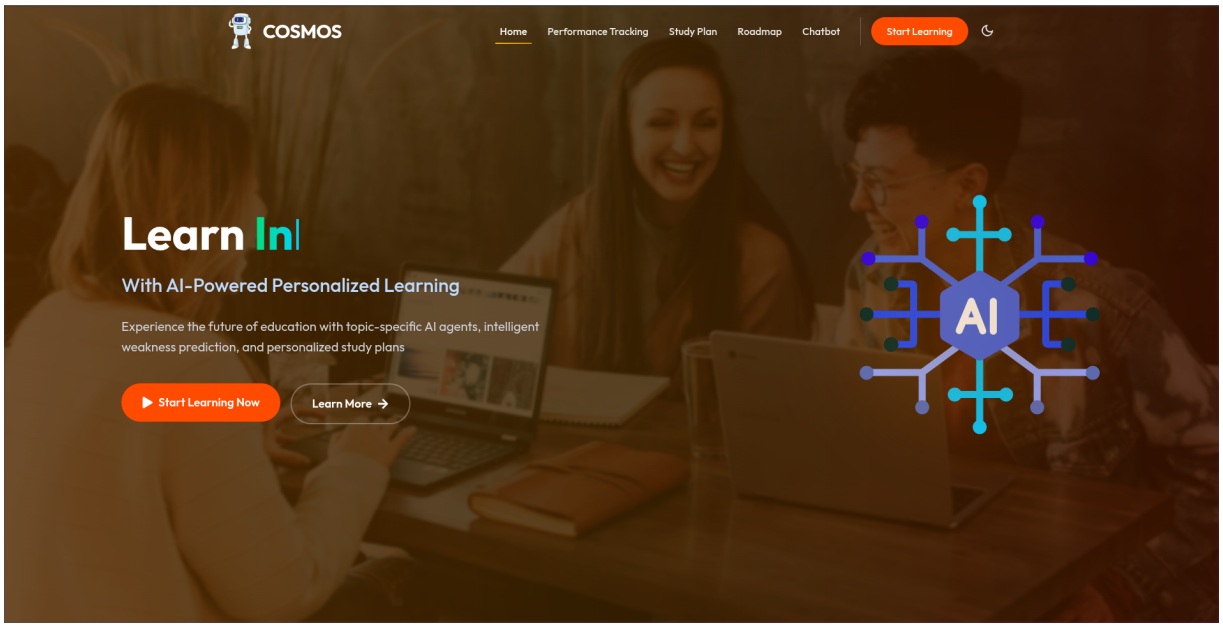


Figure 3.4: Landing Page

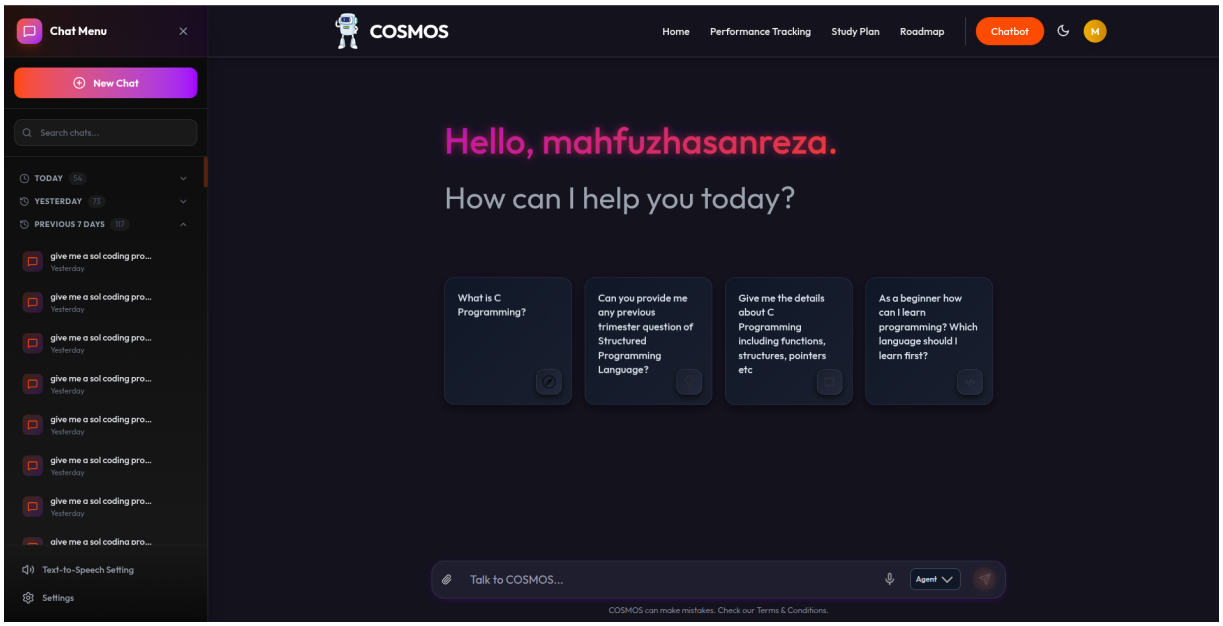


Figure 3.5: Chatbot UI

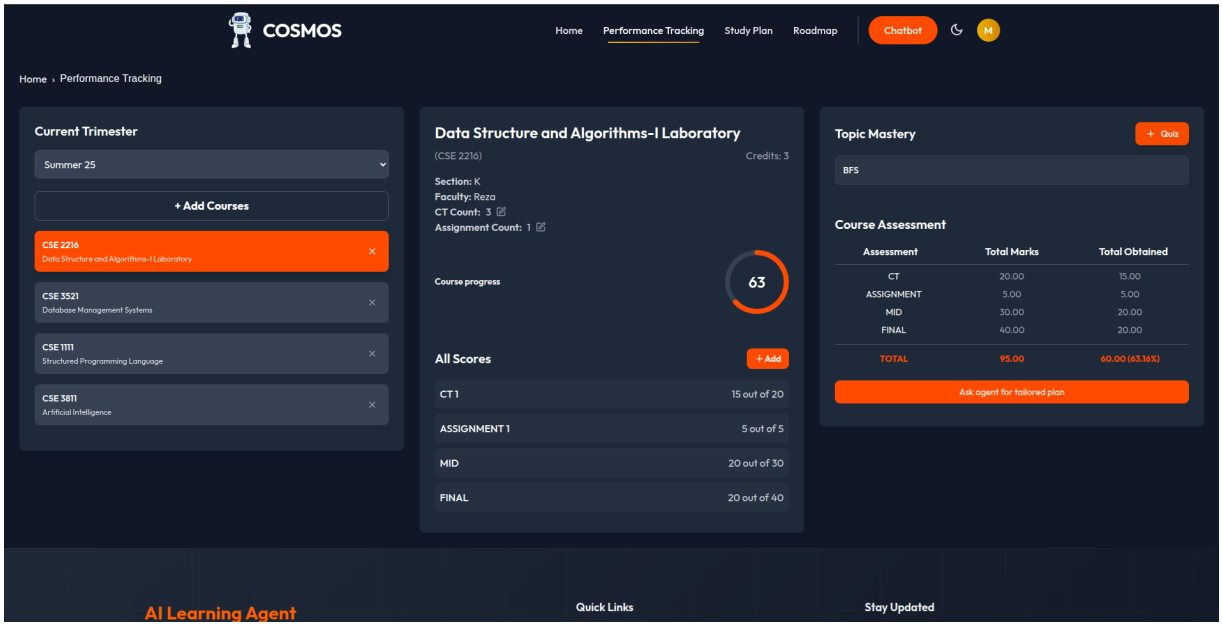


Figure 3.6: performance Tracking UI

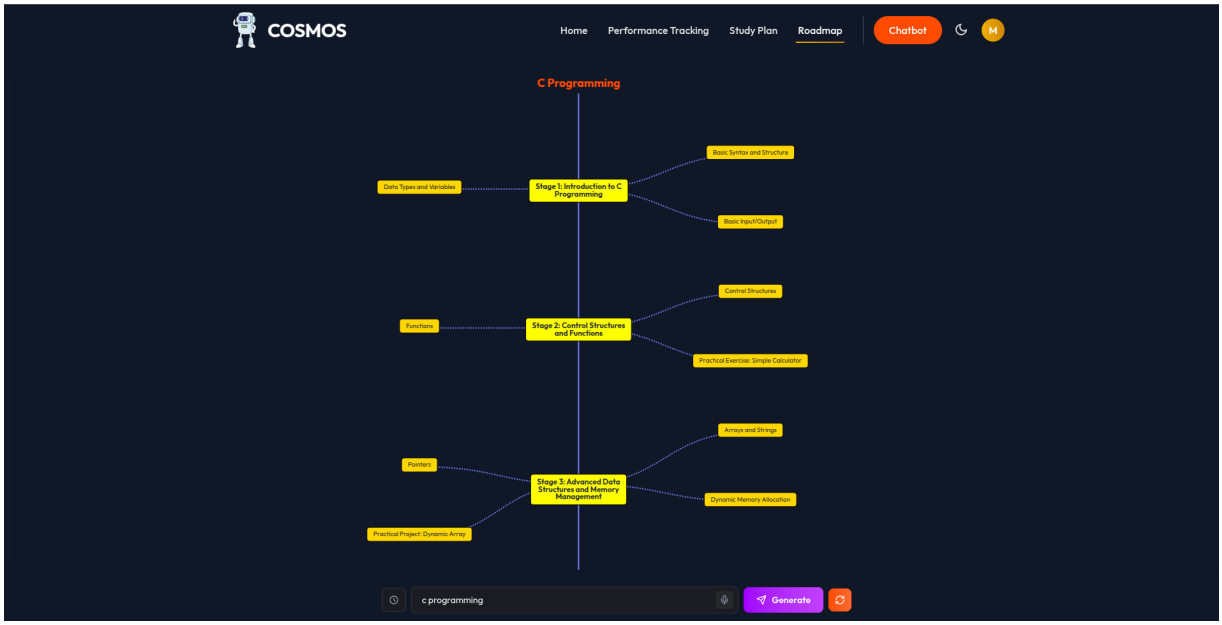


Figure 3.7: Roadmap UI

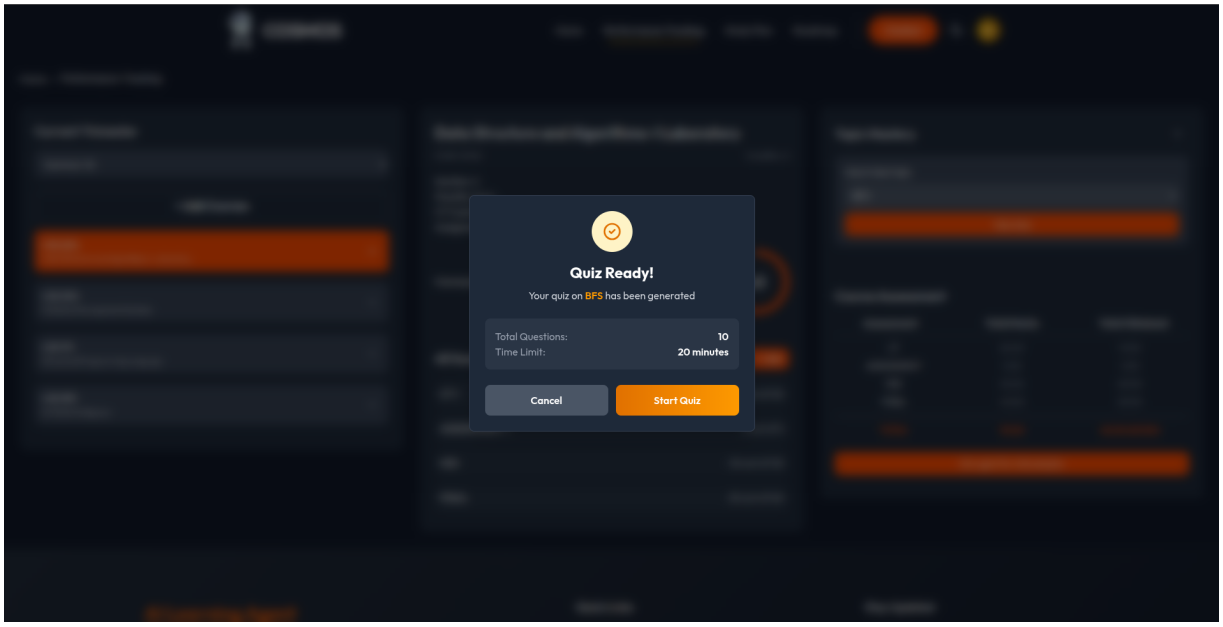


Figure 3.8: Quiz UI

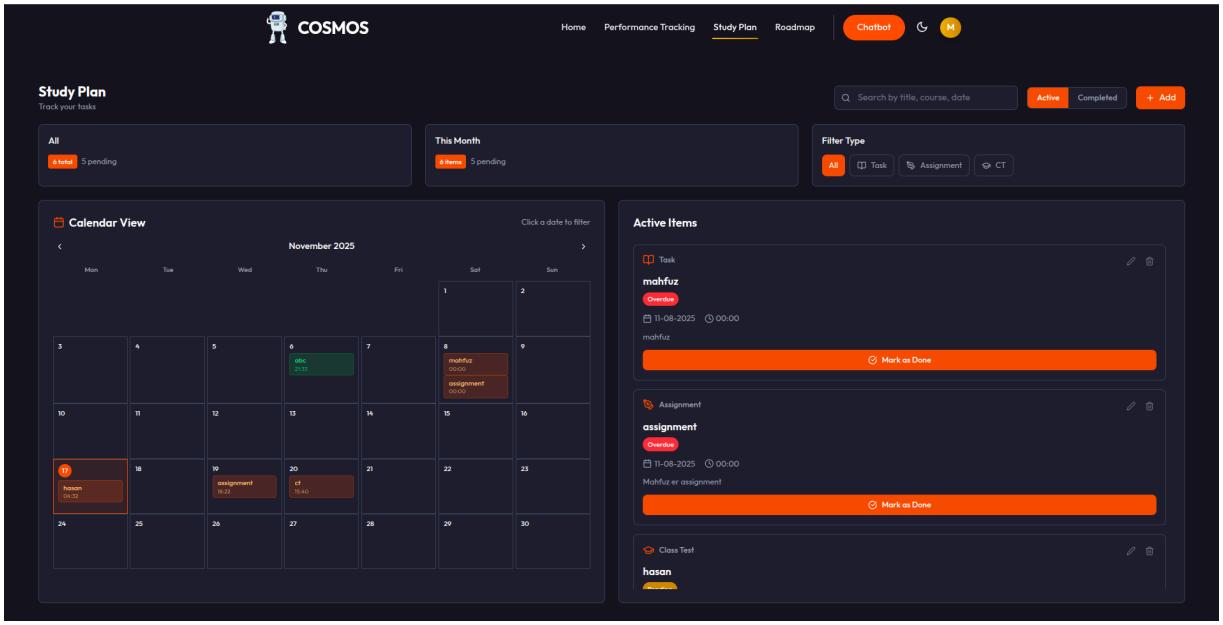


Figure 3.9: Study Planner UI

UI Design for Admin

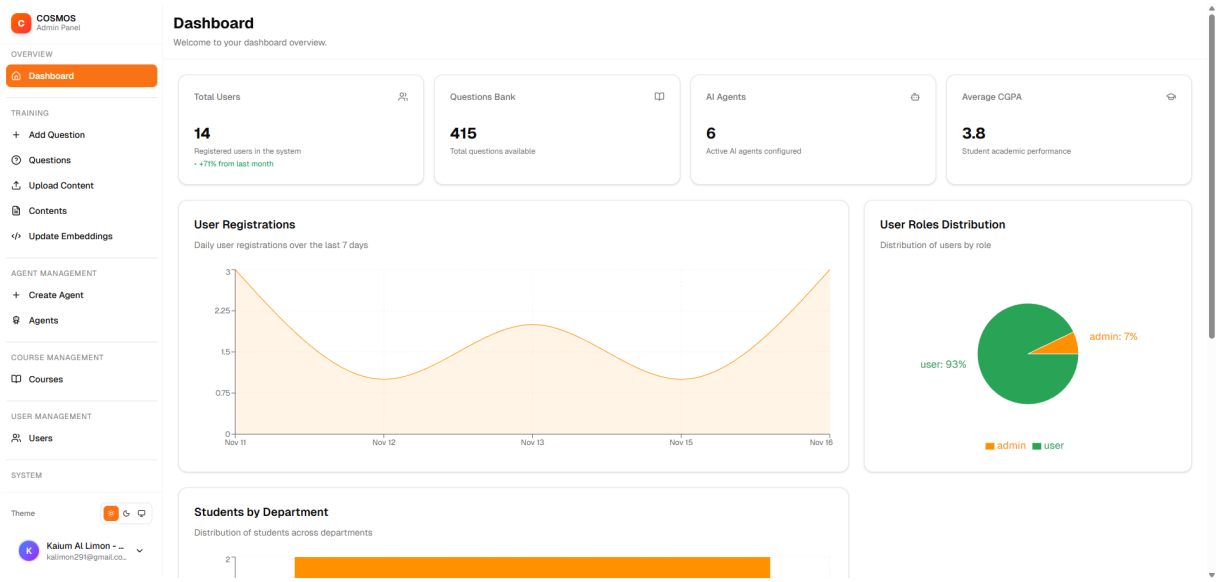


Figure 3.10: Dashboard

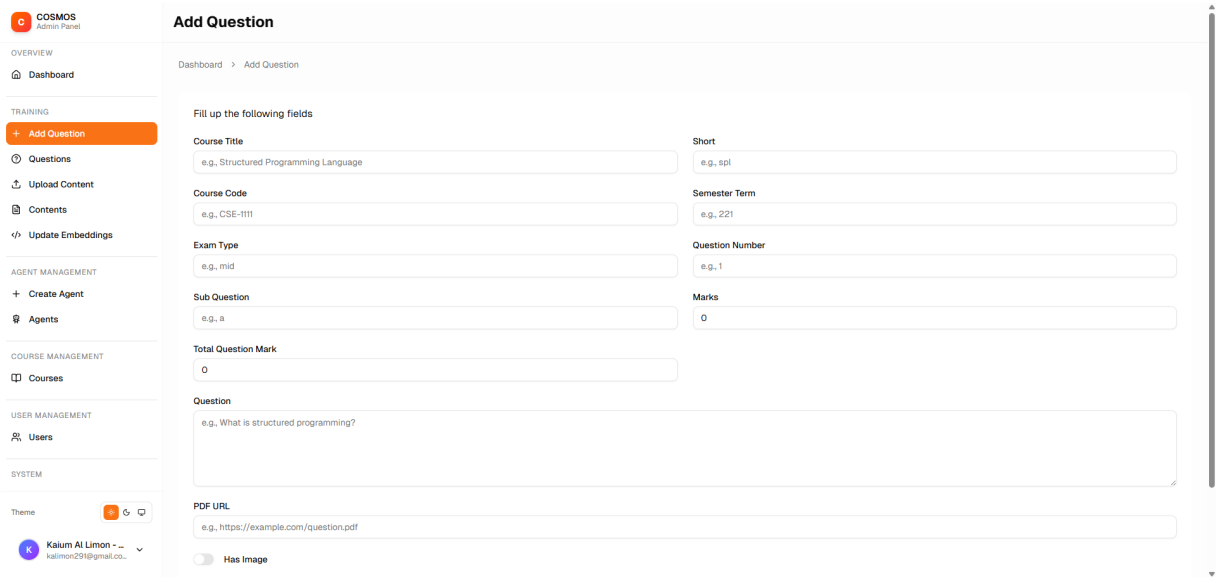


Figure 3.11: Add Question

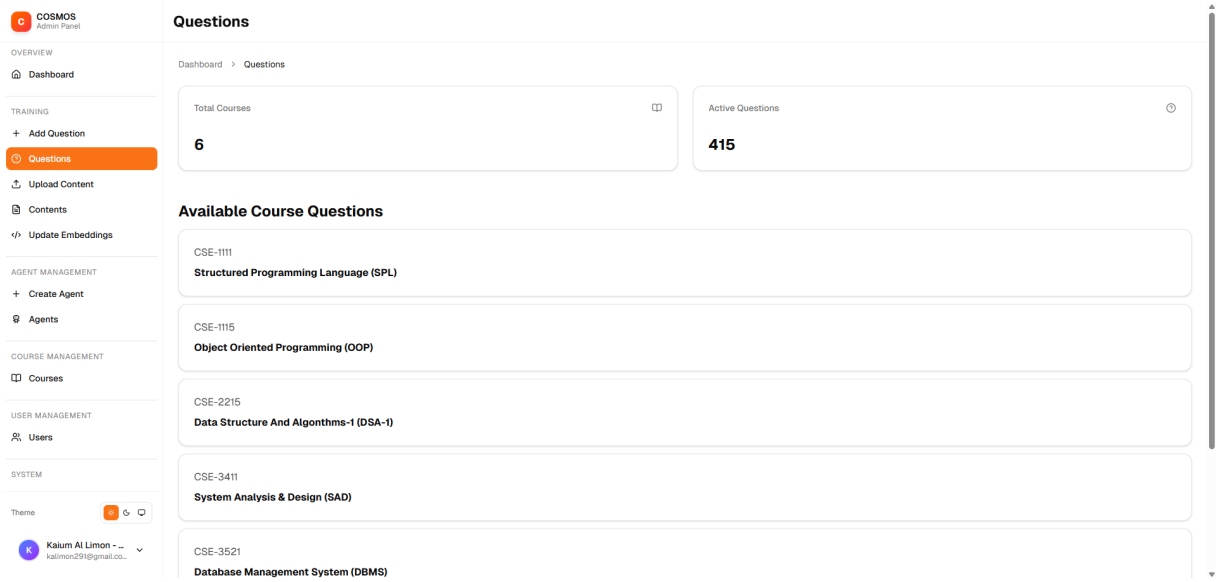


Figure 3.12: Uploaded Question

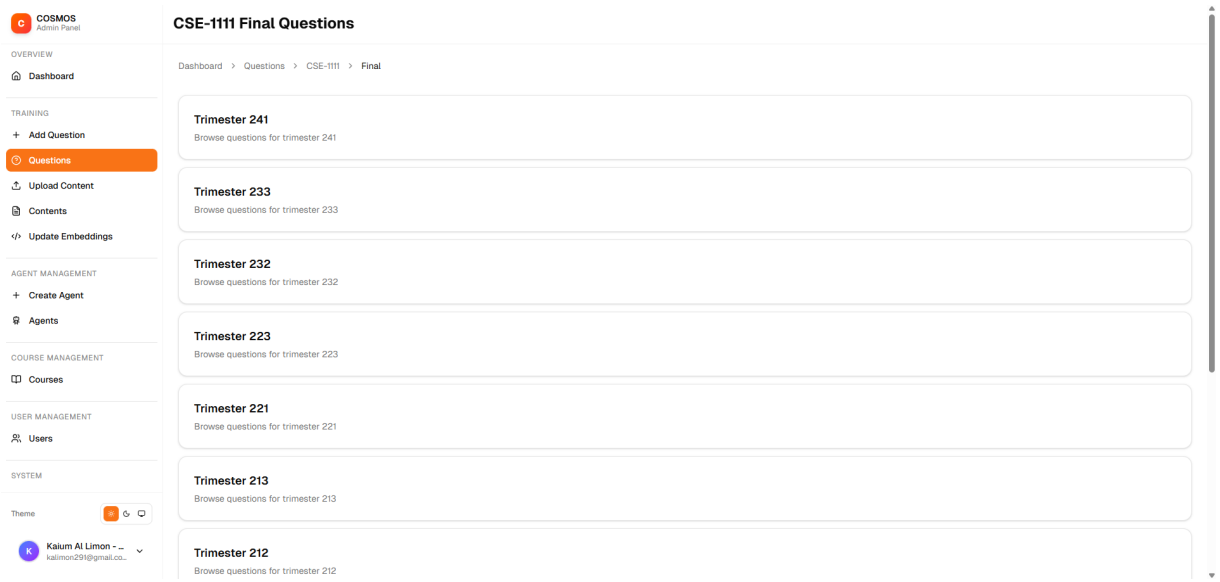


Figure 3.13: Course wise question

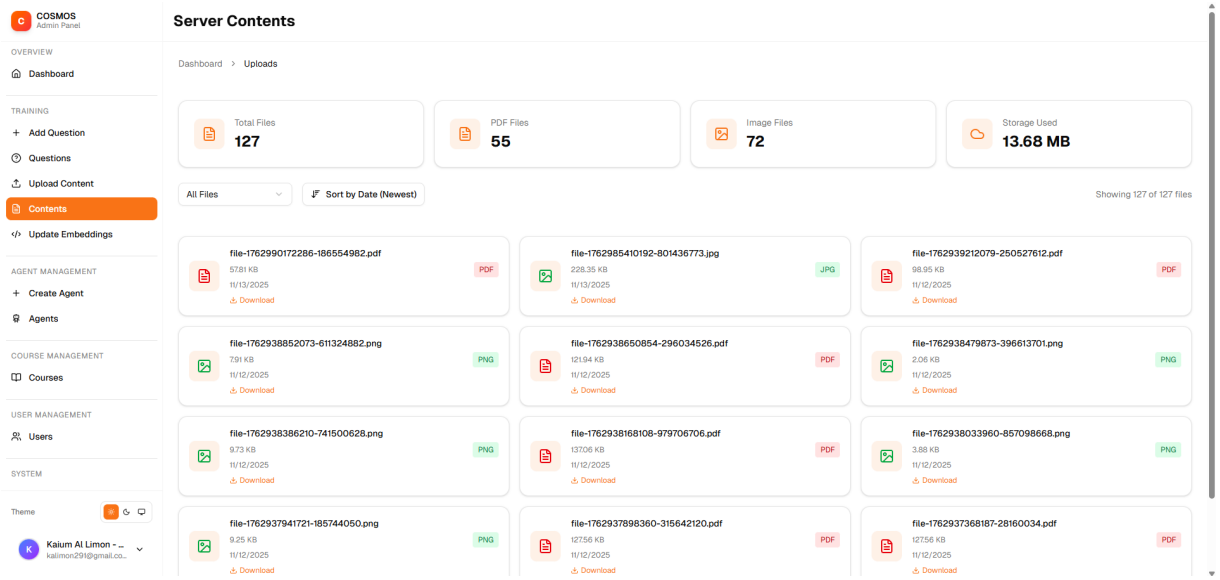


Figure 3.14: Server Content

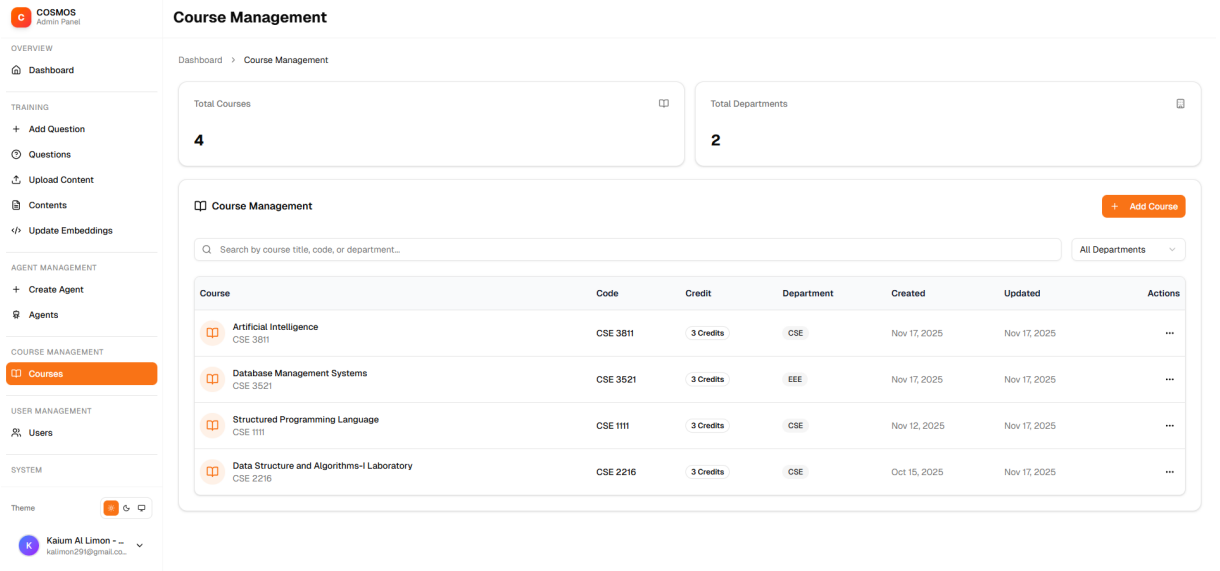


Figure 3.15: Course Management

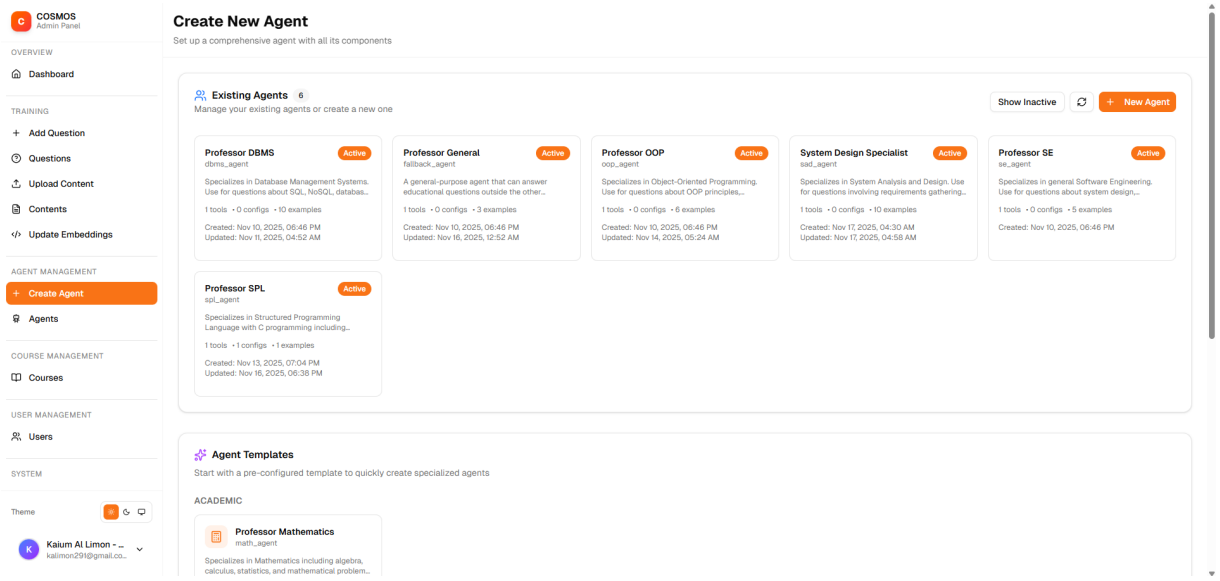


Figure 3.16: New Agent

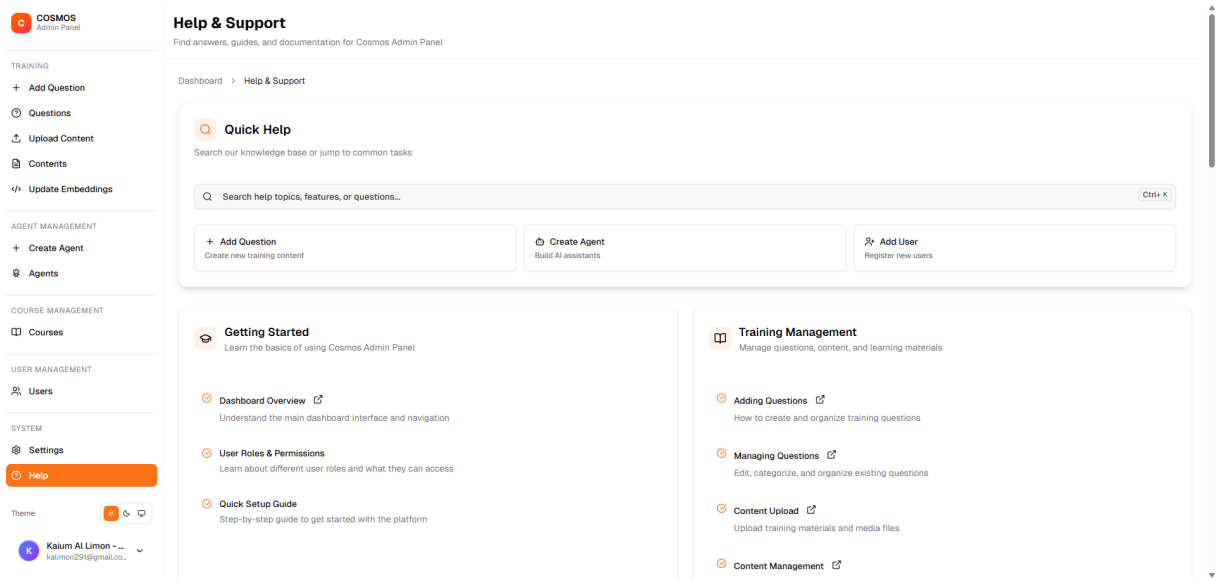


Figure 3.17: Help and Support

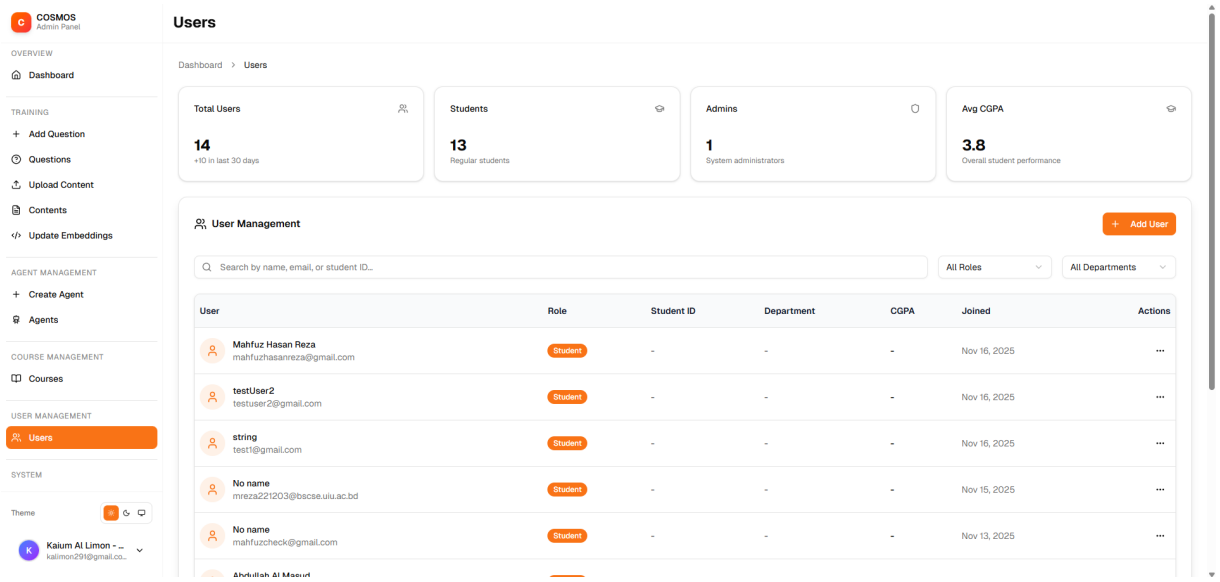


Figure 3.18: User Manage

3.2 Detailed Methodology and Design

3.2.1 Methodology: Multi-Agent LLM Architecture

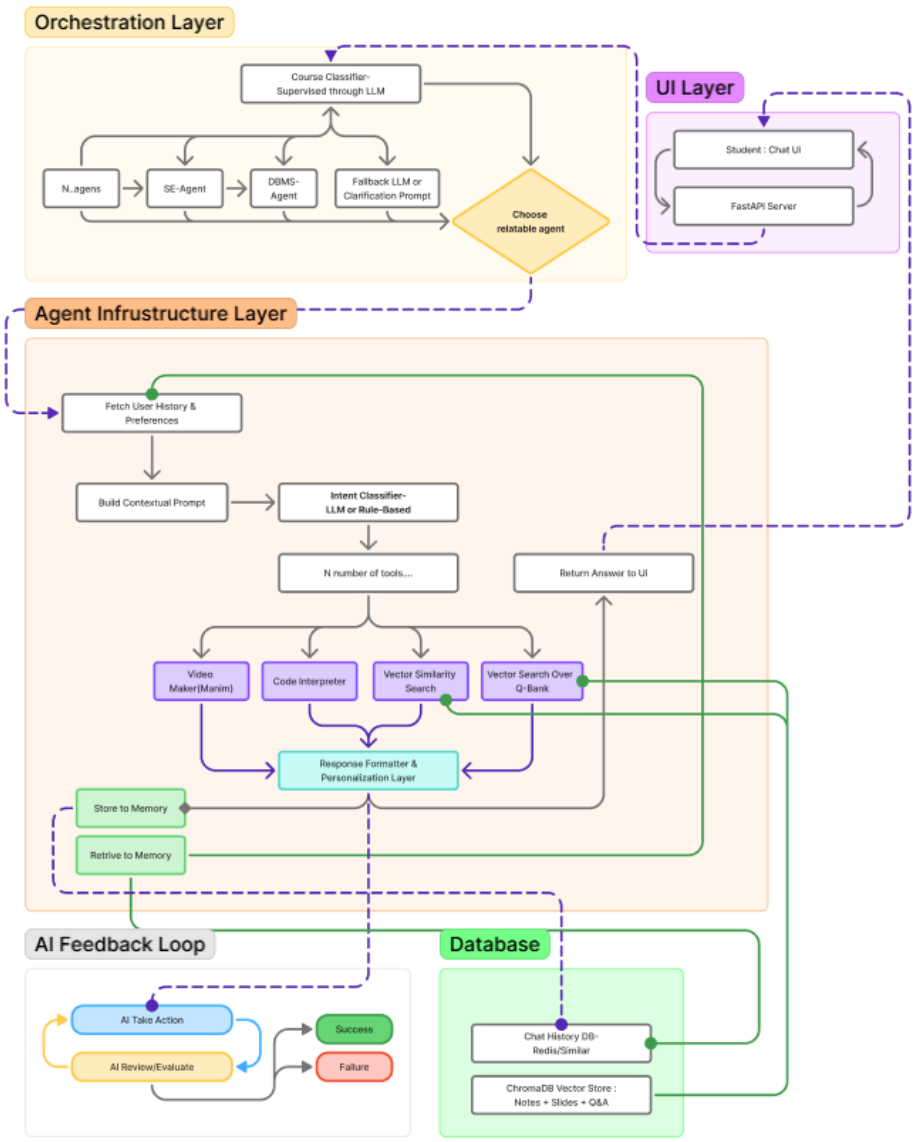


Figure 3.19: System Pipeline of COSMOS-ITS

Justification for Design Choices

Multi-Agent vs. Single-Agent Chosen: Hierarchical multi-agent with specialized agents (SE-Agent, OMM-Agent, Classification Agent)

Multi-agent systems outperform single-agent approaches for complex tasks requiring diverse expertise. Specialization improves accuracy, parallelization reduces latency, and modularity enables easier scaling compared to monolithic models.

LLM-Based Routing vs. Rule-Based **Chosen:** Supervised LLM course classifier

LLM-based routers adapt to novel and paraphrased queries, whereas rule-based systems are brittle and require constant maintenance.

Agentic vs. Deterministic Pipeline **Chosen:** Dynamic agent infrastructure with adaptive tool selection

Agentic systems allow agents to decide which tools to invoke and how many reasoning steps to take.

Memory and RAG vs. Static Prompts **Chosen:** It Fetch user history and contextual prompt building

Memory-augmented personalization can scale to millions of users while simultaneously reducing token overhead.

Hybrid (BM25 and Vector) vs. Vector Search Alone **Chosen:** Combine lexical and semantic retrieval

Hybrid retrieval captures both exact keyword matches and semantic relevance (vectors).

In-Memory and Persistent DB vs. Database-Only **Chosen:** Caching layer with persistent ChromaDB storage

In-memory caching achieves low latency, persistent storage ensures durability.

Multi-Tier Fallback vs. Single Provider **Chosen:** Fallback LLM/Classification Agent with retry logic

Multiple fallback system prevent single points of failure.

Adaptive Response Generation vs. Templates **Chosen:** Personalized response formulation layer

Adaptive generation connect s user history, preferences, and contextual prompts.

Feedback Loop vs. Static System **Chosen:** AI feedback loop tracking success/failure

Continuous learning from real-world interaction optimizes agent selection and personalisation.

Summary Table of Design Decisions

Decision	**Chosen**	**Rejected**	**Rationale**
Agent Model	Multi-agent hierarchical	Single monolithic	Specialization, parallelization, scalability
Routing	LLM-supervised	Rule-based if-then	Flexibility, adaptability to novel queries
Execution	Agentic dynamic	Deterministic fixed pipeline	Handles unanticipated complexity
Personalization	Memory + RAG	Static prompts	Scales without token explosion
Retrieval	Hybrid BM25 + vector	Vector-only	Balances precision and semantics
Caching	In-memory + persistent	DB-only	Low latency with durability
Reliability	Multi-tier fallback	Single provider	Graceful degradation
Response	Adaptive generation	Templates	User-specific, contextual output
Learning	Feedback enabled	Static system	Continuous improvement

Table 3.1: Summary Table of Design Decisions of COSMOS-ITS

3.3 Project Plan

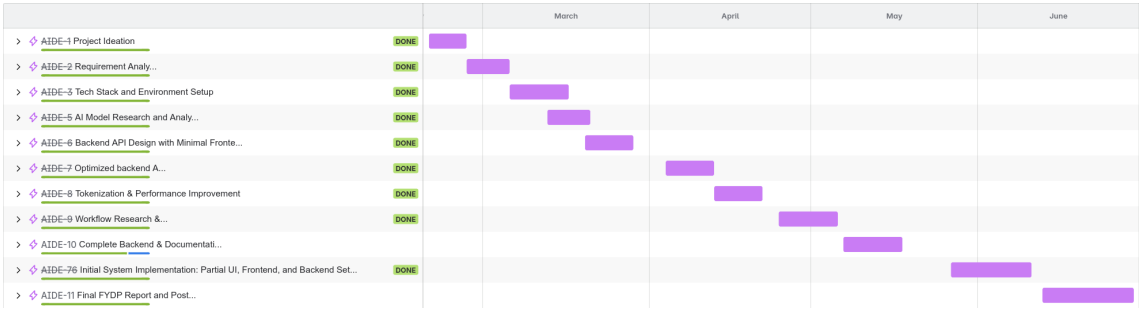


Figure 3.20: Project Plan

Month	Weeks	Activities
1	1–2	Get familiar with Intelligent Tutoring Systems (ITS) literature and multi-agent learning frameworks.
1	3–4	Enhance understanding of personalized learning algorithms, assessment models, and student performance reporting.
2	5–6	Start the design of COSMOS-ITS architecture; discover modules (UI, Orchestration, Agents, Databases).
2	7–8	Setup development environment; bootstrap the FastAPI backend, LangGraph orchestration, and database schema.
3	9–10	Add most of the chat interface, but without actual interaction between users; hook it up to FastAPI for rudimentary query routing.
3	11–12	Create course classifier (supervised + agent routing); set up Redis and Pinecone connection.
4	13–14	Develop topic-wise agents (e.g., SE Agent, DBMS Agent, DSA Agent); integrate agents with course contents.
4	15–16	Combine vector embeddings for queries of UIU.
5	17–18	Generate a customized learning path generator; map the path nodes and create content on its nodes.
5	19–20	Provide caching to LLM, making the responses faster.
6	21–22	Create assessment and weakness routings, scoring rules, and student groups; define algorithms for implementation.
6	23–24	Deploy performance tracking for CT/exam marks entry and analytics; evaluate dashboard for accuracy.
7	25–26	Build quiz generator with LLM; persist quiz scores to student performance data.

7	27–28	Link quiz results to weakness prediction and learning path updates; enhance feedback loops; create study plan module (tasking, monitoring, reminder); test usability.
8	29–30	Perform full system integration testing and load testing; verify agent orchestration, routing, and data flow.
8	31–32	Complete documentation; polish UI/UX; gather supervisor feedback and write project report.

Table 3.2: Table of Project Planning of COSMOS-ITS

3.4 Task Allocation

Projects / COSMOS / AIDE-1

1

Project Ideation

+ Add

Description

Brainstorm and finalize the core idea of the project. Define the problem statement, target audience, potential impact, and initial feature set. Create a value proposition and gather preliminary feedback.

Child work items

100% Done

Type	Key	Summary	Priority	Assignee	Status
☑	~~AIDE-21~~	Validate new idea with initial user or stakeholder feedback.	= Medium	Musfiqur Rahman	DONE
☑	~~AIDE-20~~	Explore new problem domain or direction (could involve redoing...	= Medium	Kaium Al Limon	DONE
☑	~~AIDE-19~~	Justify the decision to pivot (with stakeholder or supervisor notes).	= Medium	Mahfuz Hasan Reza	DONE
☑	~~AIDE-18~~	Document limitations or drawbacks of the initial idea.	= Medium	Kamrul Islam Arnob	DONE
☑	~~AIDE-17~~	Conduct team brainstorming session for problem domains.	= Medium	Abdullah Al Masud	DONE
☑	~~AIDE-12~~	Meditouch : Research about healthcare system	= Medium	Musfiqur Rahman	DONE
☑	~~AIDE-13~~	Brainstorm and shortlist 3 potential project ideas.	= Medium	Abdullah Al Masud	DONE
☑	~~AIDE-14~~	Conduct feasibility study for each idea.	= Medium	Kamrul Islam Arnob	DONE
☑	~~AIDE-15~~	Define high-level goals and intended impact.	= Medium	Mahfuz Hasan Reza	DONE
☑	~~AIDE-16~~	Prepare initial project pitch (for supervisor/mentor approval).	= Medium	Sadik Mahmud	DONE

Figure 3.21: Project Ideation

Projects / COSMOS / AIDE-2

1

Requirement Analysis

+ Add

Description

Gather and document functional and non-functional requirements. Engage with stakeholders to refine needs. Develop use cases, user stories, and system requirements specifications.

Child work items

100% Done

Type	Key	Summary	Priority	Assignee	Status
☑	~~AIDE-25~~	Finalize and make report	= Medium	Kamrul Islam Arnob	DONE
☑	~~AIDE-24~~	Create detailed use cases or user stories.	= Medium	Mahfuz Hasan Reza	DONE
☑	~~AIDE-23~~	Identify functional requirements (what the system should do).	= Medium	Sadik Mahmud	DONE
☑	~~AIDE-22~~	Identify non-functional requirements (what the system should do).	= Medium	Abdullah Al Masud	DONE

Figure 3.22: Requirement Analysis

Projects / COSMOS / AIDE-3

1

Tech Stack and Environment Setup

+ Add

Description

Select appropriate technologies for frontend, backend, database, and AI components. Set up development environments, version control (e.g., Git), and continuous integration workflows.

Child work items

100% Done

Type	Key	Summary	Priority	Assignee	Status
☑	~~AIDE-26~~	Evaluate potential tech stacks (Node.js, Python, MongoDB, etc.).	= Medium	Abdullah Al Masud	DONE
☑	~~AIDE-27~~	Decide on final backend, frontend, and database technologies.	= Medium	Kaium Al Limon	DONE
☑	~~AIDE-28~~	Set up project repository with folder structure.	= Medium	Kaium Al Limon	DONE
☑	~~AIDE-29~~	Configure development and deployment environments.	= Medium	Kamrul Islam Arnob	DONE
☑	~~AIDE-75~~	Set up client side project repository with folder structure.	= Medium	Mahfuz Hasan Reza	DONE
☑	~~AIDE-30~~	Establish version control and contribution guidelines (Git/GitHub).	= Medium	Mahfuz Hasan Reza	DONE

Figure 3.23: Tech Stack and Environment Setup

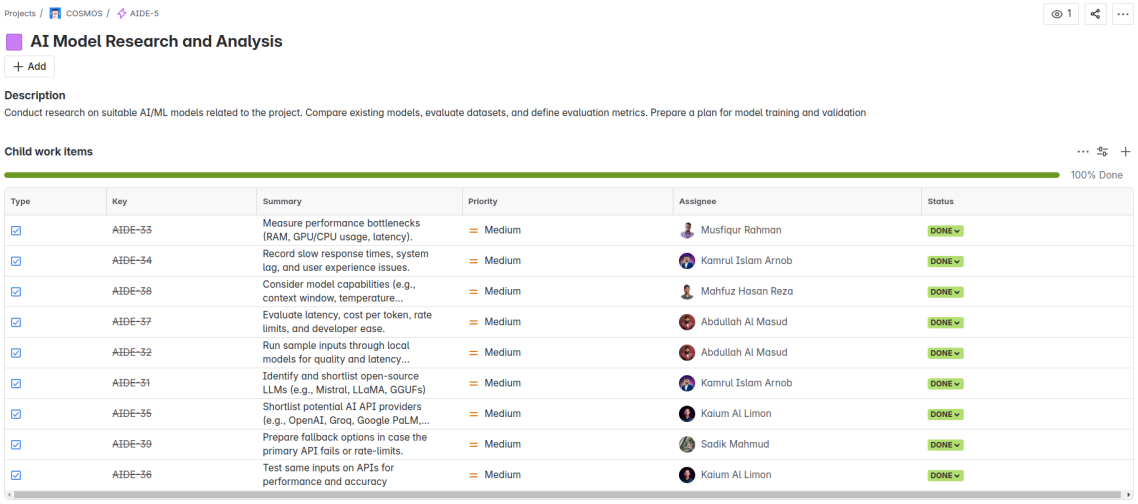


Figure 3.24: AI Model Research and Analysis

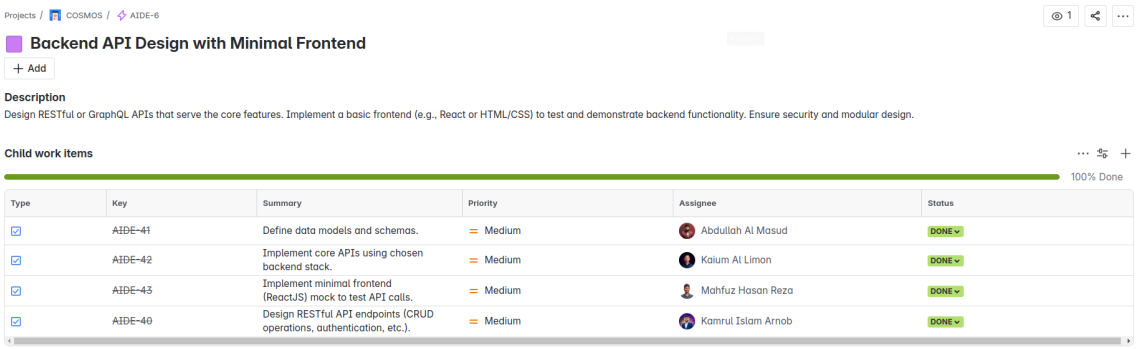


Figure 3.25: Backend AI Design with Minimal Frontend

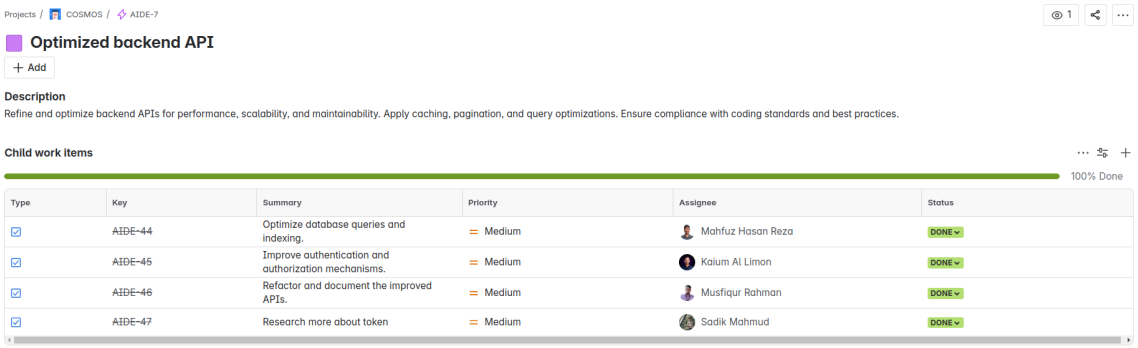


Figure 3.26: Optimized Backend API

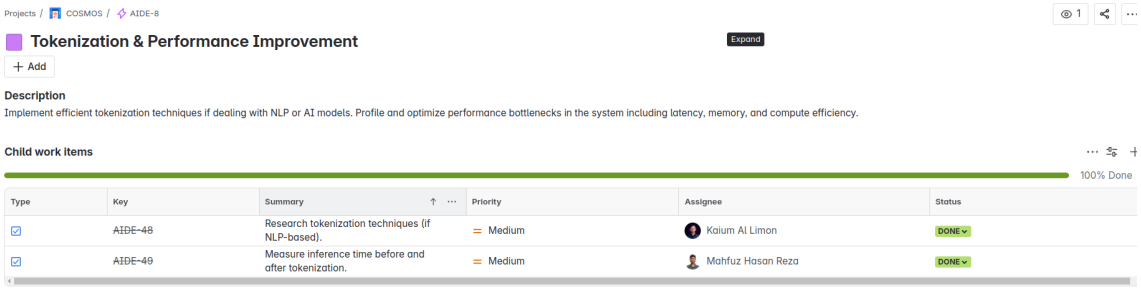


Figure 3.27: Tokenization and Performance Improvement

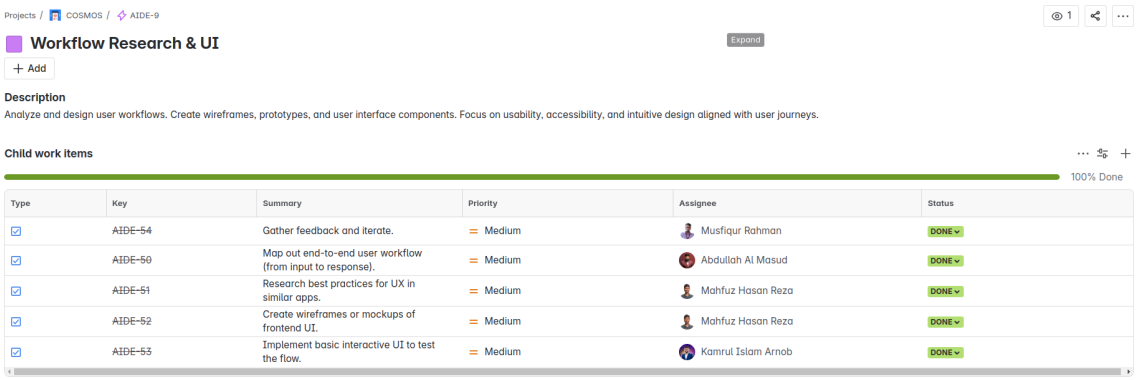


Figure 3.28: Workflow Research and UI

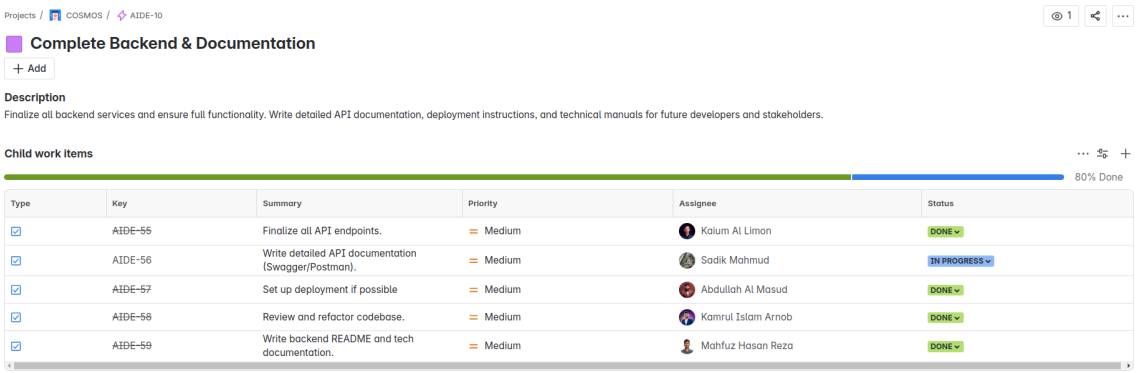


Figure 3.29: Complete Backend and Documentation

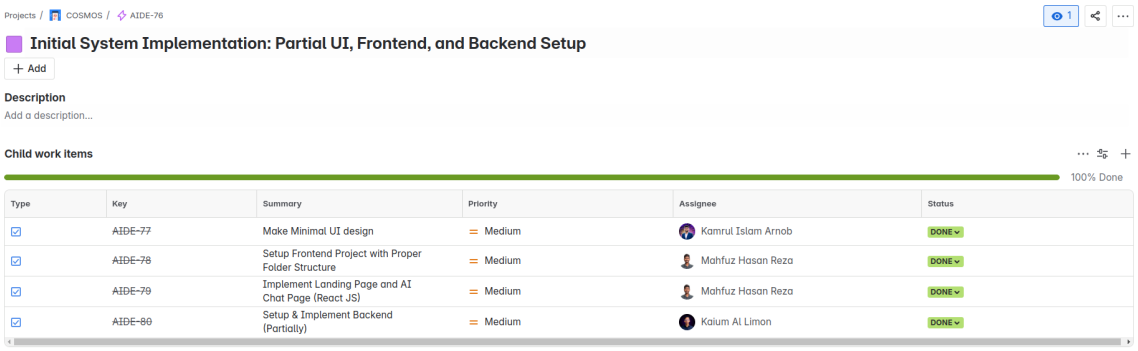


Figure 3.30: Initial System Implementation

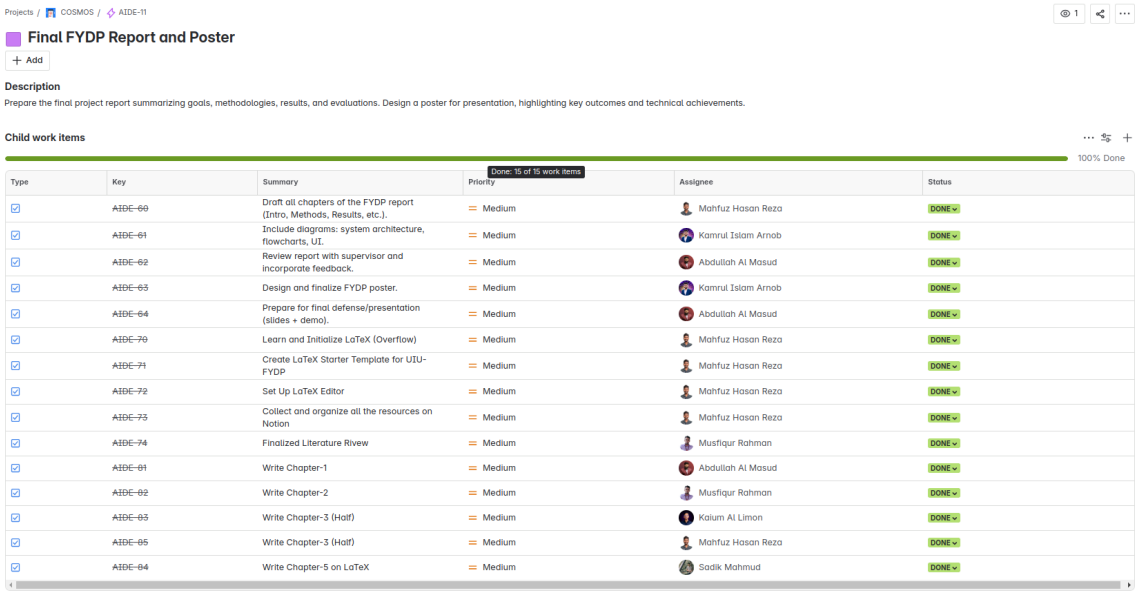


Figure 3.31: Report and Poster for FYDP-1

3.5 Summary

To ensure the effective development and timely delivery of the COSMOS-ITS system, the tasks were strategically divided into frontend, backend, and integration components. In the initial phase, our focus was placed on partial implementation, with responsibilities allocated based on expertise and project priorities.

Chapter 4

Implementation and Results

4.1 Environment Setup

A well-organized development environment was established to enable the **COSMOS-ITS** system and support multi-agent orchestration, RAG (Retrieval-Augmented Generation), vector databases, and cloud-based authentication. This environment allows the backend server, the orchestrating modules, and front-end interfaces to work together in a highly consistent fashion. The subsequent sections present all necessary setup steps from software installation to dependency configuration, environment variable management, and client-side preparation.

Software Installation

The development environment requires foundational tools to initialize and manage the project. The following software was installed:

- Git – Version control and project cloning
- VS Code – Primary code editor
- Postman – API testing tool
- Python 3.x – Backend development
- Node.js & NPM – Running client/admin panels

These tools establish the base platform needed to run the FastAPI backend, LangGraph orchestration layer, and React/Vite-based user interface.

Project Backend Setup

The server environment was initialized after installing the necessary tools.

Clone the Server Repository

```
git clone <project-url>
```

Create and Configure Environment Variables An `.env` file was created containing all required API keys, authentication credentials, and database connection URLs:

```
# LLM & Service API Keys
PINECONE_API_KEY="PINECONE_API_KEY"
OPENAI_API_KEY="OPENAI_API_KEY"

# Supabase Credentials
SUPABASE_URL="SUPABASE_PROJECT_URL"
SUPABASE_KEY="SUPABASE_ANON_KEY"
SERVICE_ROLE="SUPABASE_PROJECT_SERVICE_ROLE_KEY"

# Test User Credentials
TEST_USER_EMAIL="kalimon291@gmail.com"
TEST_USER_PASSWORD="TEST_USER_PASSWORD_FROM_SUPABASE_AUTH"

# PostgreSQL Database Connection
POSTGRES_DB_URL="POSTGRE_DB_CONNECTION_FROM_SUPABASE_PROJECT"
```

These variables allow the backend to authenticate users, connect to Supabase, and interact with Pinecone Cloud for vector search.

Virtual Environment Setup

A Python virtual environment was created to isolate dependencies:

```
python -m venv venv
```

Activation (PowerShell):

```
.\venv\Scripts\Activate.ps1
```

If PowerShell raises security issues:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

To deactivate:

```
deactivate
```

Installing Python Dependencies

The backend relies on several libraries that support:

- FastAPI backend
- RAG pipelines
- LangGraph-based agent orchestration
- Document parsing
- Database connectivity
- Vector embedding & retrieval

Dependencies were installed using:

```
pip install -r requirements.txt
```

Key libraries used:

- **Core Frameworks:** fastapi, uvicorn, langgraph, langchain
- **LLM Integrations:** langchain-google-genai, langchain-groq, langchain-pinecone
- **Database & RAG:** psycopg2-binary, supabase, pgvector, pypdf
- **Authentication:** asyncpg
- **Tools & Utilities:** python-dotenv, tiktoken, pydantic
- **Frontend Client Integration:** streamlit, sseclient-py, sse-starlette

Running the Backend Server

After installing all dependencies, start the backend with:

```
uvicorn src.main:app --reload
```

This launches the FastAPI server in development mode with automatic reload.

Client/Admin Panel Setup

The user interface and admin dashboard were implemented using Node.js.

Clone the Client/Admin Repositories

```
git clone <client-url>
git clone <admin-panel-url>
```

Install Node Dependencies

```
npm install
```

Run the Development Server

```
npm run dev
```

This initializes both the student panel and admin dashboard, enabling real-time interaction with the backend.

Cloud & Database Environment

COSMOS-ITS integrates multiple cloud-based resources:

Supabase:

- User authentication
- User metadata storage
- PostgreSQL database with pgvector support

Pinecone Cloud:

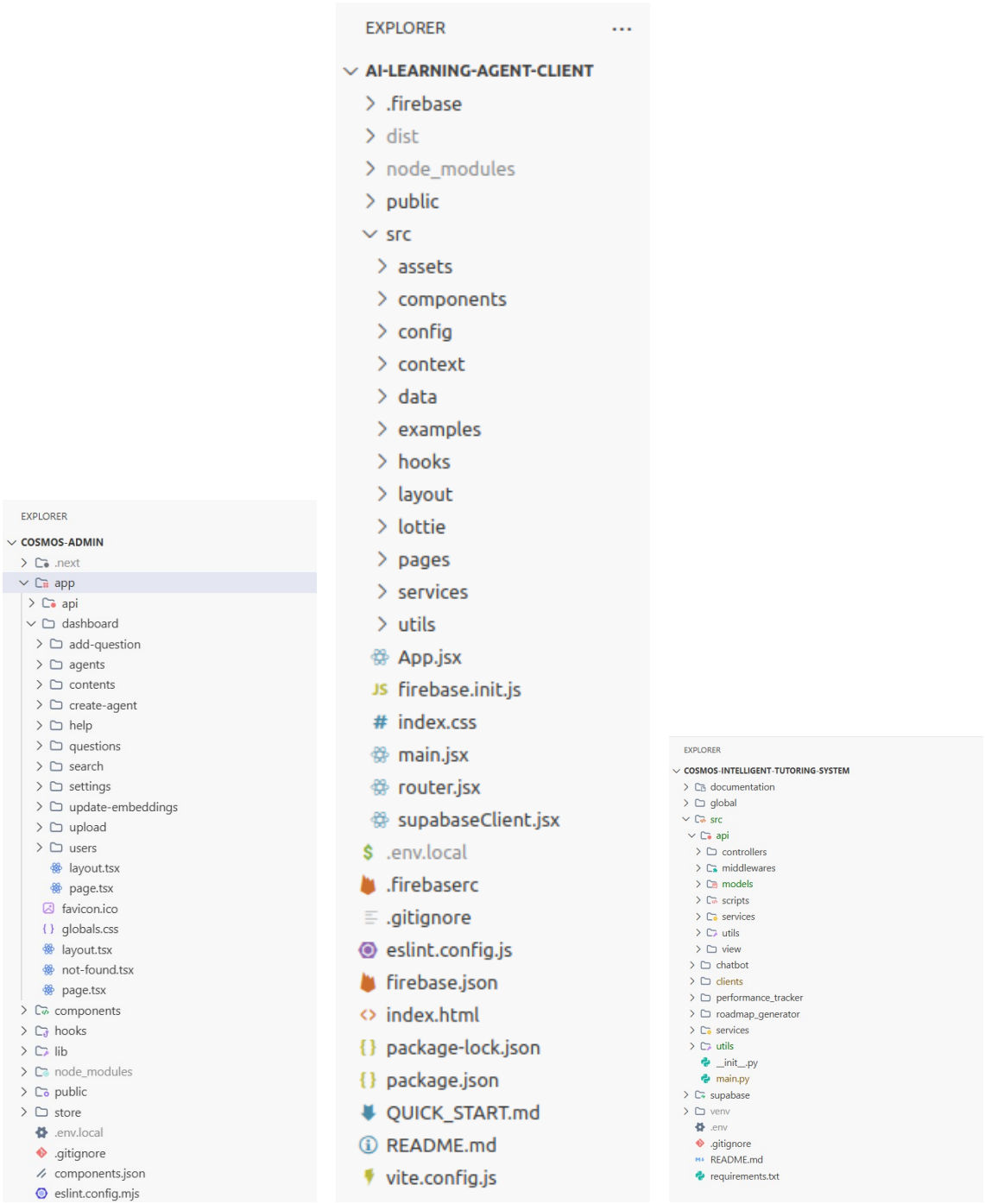
- Stores course materials as embeddings
- Supports fast semantic search
- Ensures consistency of vector indexing

All services are accessed securely via API keys stored in `.env`.

Validation of the Environment

After installation, the environment was tested via:

- API connectivity checks using Postman
- Verifying Supabase database connections
- Pinecone index initialization and vector upserts
- Stress tests on local server (multi-search, multi-agents)
- Frontend-backend communication via real chat



(a) Admin Dashboard (Next.js) (b) Client UI (React/Vite) (c) Backend Core (Python)

Figure 4.1: Side-by-Side Admin and Cliant Repository Folder Structures

4.2 Testing and Evaluation

Performance and reliability of the **COSMOS-ITS** backend were tested using **Locust**, a Python-based load testing tool. The goal was to observe system behavior under multiple concurrent users while using external resources like free-tier databases and third-party LLM APIs.

Endpoints Tested

- `POST /api/v1/chats/structured`
- `GET /api/v1/performance/courses`
- `POST /api/v1/performance/generate-quiz`

Testing Parameters

- Number of users: 50
- Spawn rate: 5 users/second
- Wait time between tasks: 1–3 seconds
- Authentication: Valid JWT token included

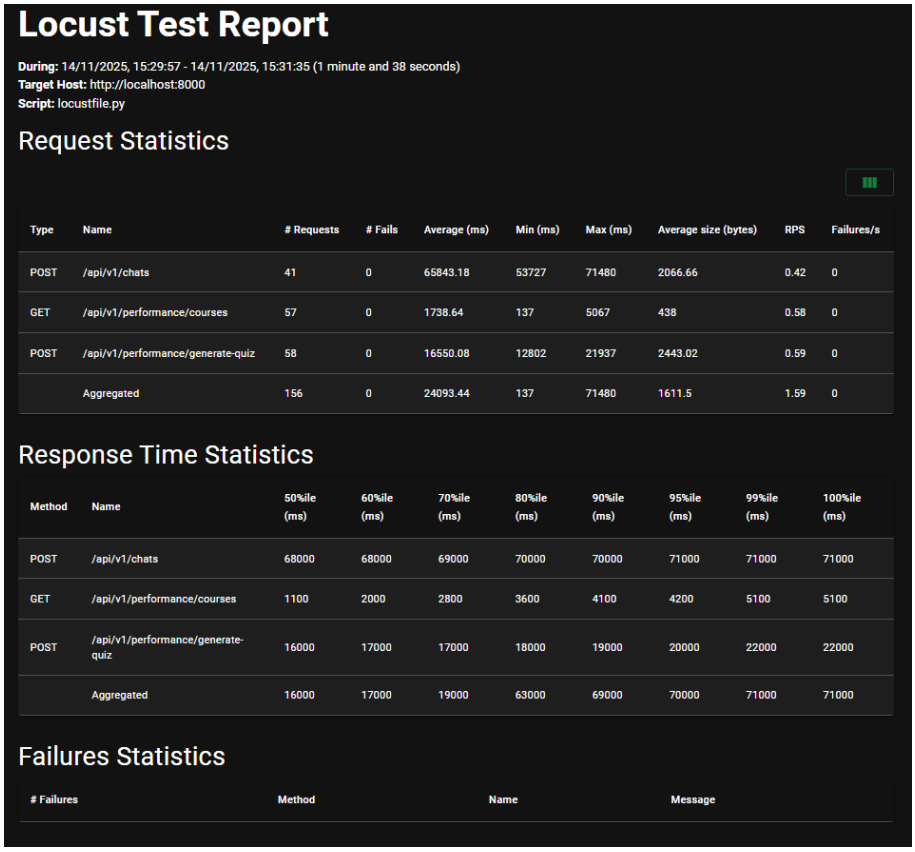
Tasks Executed per User

- Creating structured chats
- Fetching performance courses
- Generating quizzes

The Locustfile automated these actions to emulate real student interactions during peak usage.

Execution

The load test ran for 1 minute and 38 seconds, processing 156 requests without any failures. Response times, percentiles, and request throughput were automatically recorded, providing an initial performance benchmark.



Locust Test Report

During: 14/11/2025, 15:29:57 - 14/11/2025, 15:31:35 (1 minute and 38 seconds)
Target Host: http://localhost:8000
Script: locustfile.py

Request Statistics

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
POST	/api/v1/chats	41	0	65843.18	53727	71480	2066.66	0.42	0
GET	/api/v1/performance/courses	57	0	1738.64	137	5067	438	0.58	0
POST	/api/v1/performance/generate-quiz	58	0	16550.08	12802	21937	2443.02	0.59	0
	Aggregated	156	0	24093.44	137	71480	1611.5	1.59	0

Response Time Statistics

Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
POST	/api/v1/chats	68000	68000	69000	70000	70000	71000	71000	71000
GET	/api/v1/performance/courses	1100	2000	2800	3600	4100	4200	5100	5100
POST	/api/v1/performance/generate-quiz	16000	17000	17000	18000	19000	20000	22000	22000
	Aggregated	16000	17000	19000	63000	69000	70000	71000	71000

Failures Statistics

# Failures	Method	Name	Message

Figure 4.2: Load test configuration and realtime stats via Locust

4.3 Results and Discussion

1. Response Times

Structured Chat (LLM)

- Mean latency: ≈ 65,843 ms
- 95th–100th percentile: 70–71 seconds

Performance Courses

- Average latency: 1,738 ms
- 95th percentile: ≈ 4,200 ms

Quiz Generation

- Average latency: 16,550 ms
- 95th percentile: ≈ 22 seconds

2. Throughput and Stability

No errors occurred during testing. With 50 virtual users, throughput averaged around 1.5 RPS. High latency was observed for LLM-dependent endpoints.

3. Bottlenecks Identified

- The primary bottleneck is the Latency of LLM .
- as Free-tier hosting slows down DB queries during concurrent access.
- Involving both LLM and DB operations in "Endpoints" takes more time.

4. Development Stage Suitability

The system focuses on functional correctness rather than optimization. The test identifies areas for improvement prior to full deployment.

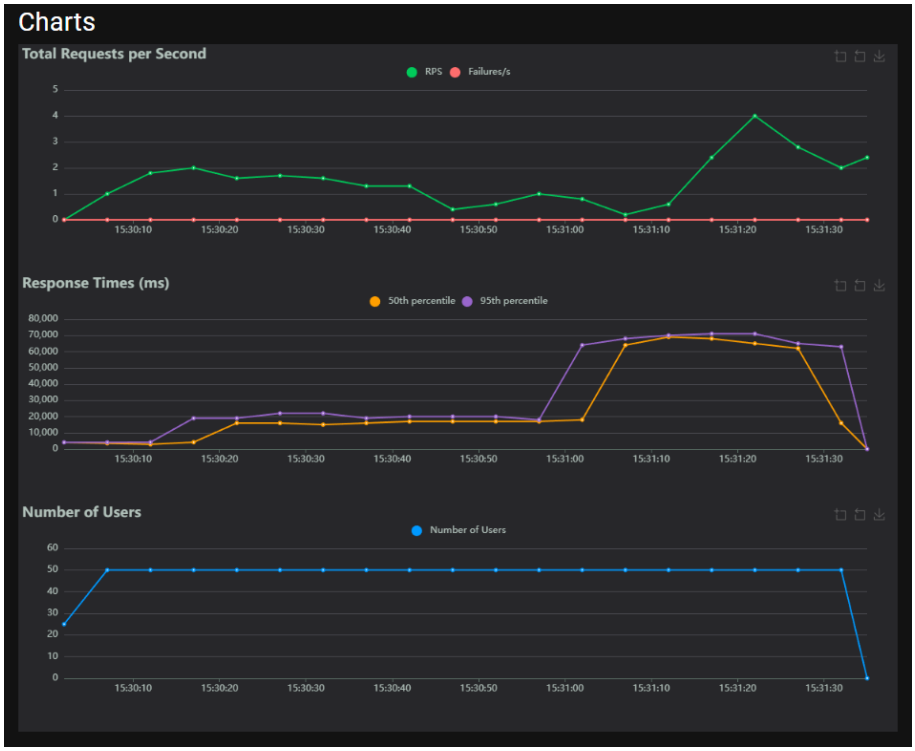


Figure 4.3: Response time and request throughput via Load Test

4.4 Summary

The testing phase provided initial performance results of the **COSMOS-ITS** backend under simulated parallel usage. With 50 virtual users and a spawn rate of 5:

- All incoming requests were handled successfully.
- No crashes or unhandled exceptions occurred.
- High response times are primarily associated with LLM-dependent endpoints.

These results align with expectations at this development stage. Future optimizations—including production-grade infrastructure, caching, and improved asynchronous processing—will reduce latency and support higher query volumes.

Chapter 5

Standards and Design Constraints

5.1 Compliance with the Standards

The **COSMOS-ITS** platform is developed using a combination of software, hardware, and communication standards to ensure reliability, scalability, and uniformity across all stages of development. Cause the system relies on multi-agent orchestration, retrieval-augmented generation (RAG), and cloud-based databases powered by vector search, complying with established data-storage and compute standards is essential for stable operation, secure data management, and consistent result generation.

This section outlines the key standards considered while system development.

At the frontend level, alongside backend and AI service conventions, we follow widely adopted industry practices. The UI is built with React and follows a modular, component-based architecture to ensure maintainability and scalability. Design and styling follow Tailwind CSS and Material UI principles, with a strong focus on responsive design and accessibility (WCAG compliance). The site is built with a PC-first approach, meaning the primary interface is optimized for desktop screens.

Additionally, the frontend build and optimization pipeline is powered by Vite, adhering to modern ES Module–based standards, resulting in faster, smaller, and more standards-compliant web applications.

5.1.1 Software Standards

The **COSMOS-ITS** infrastructure is stablish on advanced AI frameworks, backend technologies, and database-driven retrieval systems. The architecture maintains well-established software engineering practices to ensure consistency, reliability, and maintainability throughout the development works.

a. Programming and Framework Standards Python 3.x
Python serves as the core programming language due to its rich ecosystem for AI development, vector search operations, and backend data processing. The project uses:

- **FastAPI** for high-performance, asynchronous backend services.
- **LangGraph** for multi-agent orchestration.

b. LLM and RAG Standards Since **COSMOS-ITS** heavily relies on RAG:

- Embeddings are stored in canonical vector form (arrays of `float32`) compatible with ChromaDB.

The RAG pipeline follows universal best practices:

- Clean preprocessing
- Fixed-size token window chunking
- Embedding normalization
- Metadata preservation

These practices ensure reproducibility and interoperability among AI agents.

c. Database & Storage Standards The system uses:

- **Pinecone Cloud** : This following unified vector schemas, metadata conventions, and similarity search standards.
- **Supabase** :This providing ACID compliance through PostgreSQL and managing user authentication.

These standards guarantee:

- Data consistency
- Data integrity
- Fast and efficient retrieval
- Seamless interoperability across backend modules

d. API Standards All system APIs follow:

- RESTful API conventions
- OpenAPI (Swagger) documentation
- Standardized request/response schemas

These ensure unified communication across the UI layer, backend services, and the multi-agent system.

e. Frontend Standards In addition to backend and AI standards, the frontend of **COSMOS-ITS** consider to industry-aligned web development practices:

- React 19 architecture ensures modular components and maintainable code.
- PC-first responsive design provides better desktop experience while scaling smoothly to smaller devices.
- Core CSS and Tailwind CSS enable structured layouts, utility classes, and consistent styling.
- Material UI (MUI) ensures accessible (WCAG-compliant) and design-consistent UI components.
- Vite allows modern ES Module–based development, fast builds, and efficient performance.

These practices ensure a clean, accessible, scalable, and standards-compliant interface.

5.1.2 Hardware Standards

Although **COSMOS-ITS** has been primarily deployed using cloud-based resources, the system follows hardware standards that ensure smooth development, model processing, and vector retrieval operations.

a. Processor

- Multi-core CPUs (Intel Core i5 / AMD Ryzen series or better version)
- Minimum clock speed: 2.5 GHz

These are required for:

- Faster server loading while running the system locally
- Parallel query handling

b. RAM

- Minimum recommended: 16 GB
- Optimal: 32 GB+

Necessary when:

- Used for Handling many concurrent user requests
- Loading vector indexes
- Running document parsers
- To maintaining multiple agents simultaneously

c. Storage

- SSD storage,use minimum of 256 GB (with faster read/write speed)

For faster read/write ensures:

- Fast database access
- For efficient caching and local model operations

d. Network Requirements **COSMOS-ITS** deals with remote APIs (OpenAI) and Supabase:

- Stable high-speed internet
- Secure HTTPS connections

It ensures the minimized latency and safe transmission of the academic content.

5.1.3 Communication Standards

Given that the system **COSMOS-ITS** consists of multiple subsystems (UI, orchestrator, AI agents, databases, and external APIs), communication is designed according to established standards to ensure coherence, security, and interoperability.

a. API Communication

- To secure data transport used HTTPS/TLS encryption .
- Used RESTful API conventions to standardize request handling.
- Lastly usedJSON-based payloads for all modules to ensure compatibility.

This ensures that user queries, session histories, scores, and retrieved documents are transmitted securely.

b. Inter-Agent Communication Standards This multi-agent orchestration layer follows:

- Standardized message schemas
- Deterministic routing logic
- Shared context memory

These practices maintain clarity and prevent ambiguity when tasks are passed between agents.

c. Database Communication All database connections adhere to:

- Secure connections (SSL enabled)
- ACID-compliant transactions for Supabase
- Standardized vector retrieval protocols for Pinecone cloud

These measures ensure data integrity and prevent corruption or loss.

d. Frontend–Backend Communication The UI layer communicates using:

- HTTP/JSON APIs

This ensures fast, stable, and interactive chat-based tutoring experiences.

5.2 Design Constraints

5.2.1 Economic Constraint

Due to the budget constraint, and where providing an alternative cost-effective solution is inevitable than creating traditional tutoring system, **COSMOS-ITS** has been developed. This method would alleviate the need for infrastructure, while decreasing its operational cost by incorporating AI agents to automate learning and as such minimize dependence on human instructors. Moreover, the ITS system is accessible and low in cost for educational organizations and students as well, while considering developing countries context.

5.2.2 Ethical Constraint

The **COSMOS-ITS** strictly respects ethical issues as the sensitive data of students are treated and artificial intelligence is exploited to give advises. To guarantee the privacy and security of our datasets, COSMOS-ITS keeps data usage transparently by avoiding algorithmic bias and ensuring fairness for all datasets. Additionally, ethical AI principles are part of the system to develop trust and accountability in learning.

5.2.3 Social Constraint

This **COSMOS-ITS** aim that ,it is very important for social advancement by promoting equitable access to quality education. It promotes inclusive, cooperative learning experiences that connect social and geographical divides. This ITS system ensures the teamwork, digital educators and also builds the community among the students, aligning with wider educational and societal objectives.

5.2.4 Political Constraint

5.2.5 Sustainability

This **ITS** system is designed for long-term sustainability through an adaptive, energy-efficient, and low-maintenance learning platform. By aligning with Sustainable Development Goal 4 (Quality Education), COSMOS-ITS promotes ongoing, inclusive learning opportunities that can be sustained with minimal use of resources.

5.3 Cost Analysis

This table shows the monthly cost if each of 10,000 students uses 1 million tokens per month (10 billion tokens total). LLM and embedding costs are scaled up accordingly. AWS, Supabase, and other infrastructure costs may also increase in practice, but are kept constant here for a rough estimate.

Model	LLM Cost	Embedding	AWS Backend	Supabase (Pro)	Domain	SSL	Total (min, Free SB)	Total (Pro SB, all opt)	Per Student (min)	Per Student (max)
Gemini 2.5	14,000	$1,000	$76	25	$1	$0	**$15,077**	**$15,122**	**$1.51**	**1.51**
GPT-3.5 Turbo	10,000	$200	$76	25	$1	$0	**$10,277**	**$10,322**	**$1.03**	**1.03**
GPT-4o	62,500	$200	$76	25	$1	$0	**$62,777**	**$62,822**	**$6.28**	**6.28**
GPT-4.1 mini	10,000	$200	$76	25	$1	$0	**$10,277**	**$10,322**	**$1.03**	**1.03**
GPT-4.1 nano	2,500	$200	$76	25	$1	$0	**$2,777**	**$2,822**	**$0.28**	**0.28**

Figure 5.1: Cost Analysis

- **LLM Cost:** For 10 billion tokens per month (1 million tokens per student $\times$ 10,000 students).
- **Embedding Cost:** For 10 billion tokens per month.
- **AWS Backend, Supabase, Domain, SSL:** As previously detailed; note that these costs may increase in practice depending on scalability and usage.
- **Total (min, Free SB):** Includes all essential services with free Supabase and no optional tools.
- **Total (Pro SB, all opt):** Includes Supabase Pro and all optional services.
- **Per Student:** Total monthly cost divided by 10,000 students.

5.3.1 Development Cost

The development cost includes the collective effort invested in system architecture design, backend and frontend implementation, vector database integration, multi-agent workflow orchestration, UI/UX development, and cloud deployment.

For a project of this scale—a full-stack AI-driven intelligent tutoring system with multi-agent capabilities—the average software engineering rates in Bangladesh for academic-level development teams are typically as follows:

Role	**Monthly Rate (BDT)**	**Contribution**	**Cost (Q1)**
Backend Developer	50,000	API development, RAG pipeline, multi-agent orchestration, database integration	2,00,000
Frontend Developer	36,000	UI development, admin panel, client-side integration	1,44,000
AI/ML Engineer	50,000	LLM integration, embedding pipeline, vector search integration	2,00,000
DevOps/Cloud Engineer	36,000	Deployment, environment setup, Supabase and Pinecone configuration	1,44,000
QA Engineer	36,000	Manual testing, API validation, frontend QA	1,44,000

Table 5.1: Development Team Cost Breakdown

Total Team Cost/Month: BDT 2,08,000
Total Project Cost (Q1): BDT 8,32,000

5.4 Complex Engineering Problem

5.4.1 Complex Problem Solving

The development of COSMOS-ITS addresses an engineering problem that satisfies all seven characteristics of complex problem solving as defined by the Washington Accord. Table 5.2 outlines the mapping of each problem-solving dimension (P1–P7) that applies to this project.

P1: Depth of Knowledge Required

COSMOS-ITS requires in-depth knowledge of advanced AI, backend systems, and educational technology. Key knowledge areas include:

- Multi-agent orchestration (CrewAI)

Table 5.2: Mapping with complex problem solving.

P1 Dept of Knowledge	P2 Range of Conflicting Requirements	P3 Depth of Analysis	P4 Familiarity of Issues	P5 Extent of Applicable Codes	P6 Extent of Stakeholder Involvement	P7 Interdependence
√	√	√	√	√	√	√

- LLM integration (e.g., Gemini API, OpenAI)
- Intent classification and routing
- Vector similarity search with Pinecone/ChromaDB
- Backend concurrency using FastAPI
- RAG (Retrieval-Augmented Generation) pipelines

Knowledge Profile Mapping:

- WK4: Principles-based modeling and AI agent design
- WK6: Use of novel technologies like LLMs and vector DBs

These advanced domains reflect that the problem requires deep disciplinary expertise and knowledge synthesis, satisfying P1.

P2: Range of Conflicting Requirements

COSMOS-ITS involves multiple conflicting demands:

- Responsiveness vs. accuracy (real-time vs. multi-agent processing)
- Personalization vs. privacy (adaptive recommendations vs. student data protection)
- Cost vs. performance (LLM API usage vs. efficiency using fallback models)
- Tool selection conflicts (Docling, Q-bank, and Manim for overlapping queries)

These trade-offs require complex balancing and architectural decisions, satisfying P2.

P3: Depth of Analysis Required

The system does not follow a ready-made design. It requires original problem-solving through:

- Designing course-wise intelligent agents

- Creating custom classifiers for subject detection
- Modeling student weaknesses via quiz/test analytics
- Integrating multimodal inputs: quizzes, feedback, and academic data

This aligns with abstract problem decomposition and original model development, satisfying P3.

P4: Familiarity of Issues

COSMOS-ITS addresses a novel, rarely encountered educational problem:

- Building course-specific multi-agent tutoring for university-level education
- Live routing of academic queries to agents using classification logic
- Feedback loop design for AI performance refinement
- Academic data integration with learning path decisions

These challenges are not standard in engineering practice and involve innovation, satisfying P4.

P5: Extent of Applicable Codes

The project operates in areas lacking standardization:

- No established standard for educational multi-agent AI systems
- Use of LLMs in education lies in a regulatory grey area
- Systems must be GDPR/FERPA-conscious without clear software engineering blueprints

This indicates work beyond conventional codes and standards, fulfilling P5.

P6: Extent of Stakeholder Involvement

The solution involves multiple and diverse stakeholders:

- Students (study plans, quiz feedback, learning progress)
- Faculty (content validation and result monitoring)
- IT administrators (data privacy, integration, and scaling)
- Researchers (analytical insights, performance trends)

Their different needs create design conflicts and complexity in deployment, meeting the criteria for P6.

P7: Interdependence

The system exhibits high interdependence:

- Each query flows through classifier → agent → tool → formatter → feedback
- Agents depend on course-specific data vectors and tools
- Past user behavior influences future suggestions
- Tool outputs (e.g., animations or code) must align with agent outputs

This demonstrates significant modular dependencies and layered architecture, satisfying P7.

Conclusion

Through a de tailed examination and mapping with the seven dimensions of complex problem-solving (P1–P7), it is clear that COSMOS-ITS falls under the complex engineering category. This project incorporates emerging AI, deep technical understanding, diversity of stakeholders and high level interdependencies across systems — all factors that makes it technically challenging and also translates into an educationally-rich activity.

5.4.2 Engineering Activities

The COSMOS-ITS is project involves a long-term complex design process that goes beyond regular development and management procedure. They necessitate engagement with a range of technologies, the development of new types of systems and consideration to the possible social implications. Mapping of relevant engineering activity attributes according to the COSMOS-ITS system are shown in Table 5.3.

Table 5.3: Mapping with complex engineering activities.

A1 Range of resources	A2 Level of Interaction	A3 Innovation	A4 Consequences for society and environment	A5 Familiarity
√	√	√	√	√

A1: Range of Resources

COSMOS-ITS integrates multiple tools and frameworks that require diverse technical expertise, including:

- Large Language Models (LLMs) via Gemini/OpenAI APIs

- Multi-agent frameworks (e.g., CrewAI)
- Vector databases (Pinecone, ChromaDB)
- Real-time databases (Supabase)
- Manim for dynamic visualization and animation
- Backend systems with FastAPI and async concurrency

These varied and interdependent resources demand the use of high-level engineering skills across AI, software, data handling, and real-time communication domains.

A2: Level of Interaction

This system interacts with multiple stakeholders and components:

- User interfaces interact with learners, the performance and input history are storing in DB
- Based on the contex the Multi-agent systems collaborate to answer questions .
- Backend layers must coordinate between query classification, AI responses, and visualization tools
- Teachers and university IT staff interact through content upload and agent validation

All these interactions are dynamic and multi-directional, this is confirming that COSMOS-ITS exhibits high interaction complexity.

A3: Innovation

COSMOS-ITS introduces a novel educational infrastructure by combining:

- Memory-aware tutoring agents per subject
- Using real-time performance data the system able to generate automatic learning path
- AI-assisted weakness prediction and personalized remediation plans
- Adaptive tool routing (Q-bank, Manim, document agents) based on intent recognition

These are capable to represent new methods and system behaviors that are not yet mainstream in educational AI, establishing a strong basis for A3.

A4: Consequences for Society and Environment

The project involves educational data processing and personalization. Potential consequences include:

- Positive: Increased access to AI-driven, adaptive education
- Risks: Privacy issues, bias in AI agents, over-reliance on automation
- Mitigation: Use of secure backends, transparent feedback loops, and human-in-the-loop validation

Therefore, awareness of the ethical and social impacts is necessary for this project—especially in an educational environment.

A5: Familiarity

The design of COSMOS-ITS is not based on standard frameworks or off-the-shelf solutions. It requires:

- Integration of multiple LLM-based agents across domains
- Custom classifiers for academic subject routing
- Construction of feedback loops for iterative performance improvement

These are primarily unfamiliar design challenges in the context of software engineering and educational technology, which make A5 applicable.

5.5 Summary

The COSMOS-ITS project certainly meets the criteria according to the Washington Accord of a complex engineering task. Further through fine-grained mapping and justification, the approach satisfies all seven of features, including prior work experience and high inter-relation, innovation and that students have to engage in original analysis (research) as well as developing a method. This can be achieved by more advanced integration of multi-agent systems, LLM driven orchestration and individualised educational routing.

Furthermore, COSMOS-ITS targets all five engineering activity properties (A1–A5), implies a considerable variety of technical resources, high engagement with stakeholders, major design novelty and societal impact possible. The novelty of the addressed problems -memory-aware tutoring agents, dynamic study planning, feedback-driven learning loops among others- further supports this non-conventional engineering effort.

Finally, COSMOS-ITS is as much a technically challenging project as it is an educationally transformative effort. This requires interdisciplinary knowledge and thoughtful systems engineering to achieve scalability, ethical use, and institutional adaptation. The level of the design problem provides a case which is complex enough, and deep enough to be used as an example of real-world, complex engineering problem-solving.

Chapter 6

Conclusion

6.1 Summary

Combining a multi-agent orchestration approach (Section 3.1) the **COSMOS-ITS** solution represents an advancement in intelligent tutoring systems field, with RAG-based vector search to retrieve problem-related questions from the cloud. The Security and computation are offloaded to cloud servers or user-owned devices rather than relying on insecure local machines. This system is designed to provide AI learning agents that deliver tailored, in-context advice.It also provides domain-specific guidance to students while preserving reliability, robustness and academic integrity for the learning experience.

Using FastAPI, LangGraph and Pinecone,in order to achieve this a modular system. There was multiple specialized agents are implimented so that it collaborate to provide accurate responses. The Course Agent fetches relevant academic questions where Question-Bank Agent retrieves realistic scenarios, the Classifier determines student intent finally the Fallback LLM ensures responses when course-specific content is unavailable. Here, maintaing a secure session is maintained across frontend and backend modules using Supabase for authentication and PostgreSQL for storage.

A noted limitation here is the reliance on a lightweight RAG document-chunking and embedding system. Which may be simple to understand but may not provide sufficient depth for lecture note, slide, and summary retrieval. Nonetheless, the multi-agent workflow with deterministic routing and shared context memory enables consistent, structured and academically accurate responses.

Throughout development best practices of complex engineering were followed. That includes ensuring data integrity, secure communication, software quality attributes, and scalable cloud architecture. **COSMOS-ITS** effectively addresses shortcomings of existing academic support systems by providing explanations and domain-specific feedback rather than ready-made or any type of shallow answers. We conclude that **COSMOS-ITS** is a reliable and practical AI-based student assistance system for the institutions seeking an artificial intelligence-driven solution that balances accuracy and intelligent automation. .

6.2 Limitation

Though **COSMOS-ITS** provides an efficient and scalable intelligent tutoring solution, there were several identified limitations during development and evaluation. These limitations are pragmatic issues specific to large-scale RAG systems, multi-agent orchestration, and cloud-based dependencies.

Token and Context Constraints of LLMs The model gathers fine-grained explanations via LLMs and the Course Agents resamples training examples for providing labels. But transformers have a hard limit on the number of input tokens that can be handled at once. Long transcripts, conversational contexts, or question-bank interactions have to be chunked or truncated and may compromise the degree of depth and coherence in responses.

Dependency on External APIs COSMOS-ITS relies on OpenAI, Pinecone, and Supabase services. Rate limits, network latency, service failures and price fluctuations affect system performance.

Vector Retrieval Constraints The quality of entity embeddings is crucial for RAG performance. Messy course files, incomplete slides/images, or inconsistent formatting can reduce retrieval success. This affects the accuracy of the course agent's responses.

Multi-Agent Complexity Though multi-agent orchestration provides flexibility, but the LangGraph-based system is complex. The Misclassified questions, routing errors, and context violations can lead to unnecessary fallback calls or duplicate processing. Perfect coordination among agents is difficult when student queries are ambiguous.

Computational and Resource Requirements While running the multiple agents,memory and server performance demands are high ,loading embeddings and serve it to many concurrent users. Cloud services certainly mitigate the problem to some extent but if there is load higher than what we're able to scale in real-time then performance during that time will decrease.

Limited Evaluation Dataset The accuracy and reliability of the system largely depend on the content covered in the course. Additionally, the design of the questions used for testing students and the initial question bank also influence it. Real-world academic questions are more diverse, and in some marginal cases, incorrect or incomplete answers may still appear.

Security and Privacy Considerations Supabase provides authentication and secure APIs, but stricter privacy standards are required for educational management (e.g.,

FERPA/GDPR). Stronger policies are needed beyond the prototype level to ensure the protection of academic records, student identities, and dialogue logs.

User Interface Limitations Though the current roadmap generator UI is functional but the emplimentations is very basic. Real-time updates, session state management, and enhanced UI handling are the areas for improvement to achieve a production-ready project.

6.3 Future Work

While COSMOS-ITS provides a solid foundation for intelligent tutoring with multi-agent orchestration and RAG pipelines, several future advancements can make the system more appealing to academic institutions.

Some of the more significant new extensions provided by this project are: Weakness Prediction and Group Suggestion. The current system does provide analyses to student questions and personalized feedback, however, the system does not currently try to construct a custom-tailored long term weakness profile for the students. Predictive learning model Recommendation of study groups can be made for students with similar learning habits, recommendations on deficient points can be given to each student. This would change the game in group study and tutoring.

An even more significant direction is to derive a course outcome prediction model not by using simulated data but rather with real university performance data. We had actual grades of a student in classroom tests, homework assignments and quizzes and midterm examination and we wanted to train a model that can predict whether a student will fail or pass the course only looking at his/her activities until mid term. This predictability would allow COSMOS-ITS to give students early warnings, individualized advice and custom content help, making it a predictive rather than remedial learning tool.

Also, we could enrich the system by adding a Notice-Fetching Agent that scrapes or fetch official notices from university website. Next, the students can ask the chatbot for new updates, exam times and other departmental information and receive current content. Integrating this with the rest of agent workflow would extend platform use case beyond education tutoring, and become a consolidated student support services resource.

To obtain a higher quality of performance in the future, we will optimize our multi-agent pipeline to make it faster, work on better RAG retrieval mechanism and improve the UI/ UX for making interaction capability more user friendly. What is more promising are the additional extensions to vector store, along with better-organised academic materials, a video lecture summarization and voice interaction.

Generally, these suggested improvements would move COSMOS-ITS towards a more intelligent, proactive and institution integrated learning ecosystem that can offer better anticipatory guidance to students through their academic lifecycle.

References

[1] Khan Academy. Khanmigo, 2025. Accessed: 2025-06-30.

[2] Mathpresso. Qanda: Ai-based learning platform, 2025. Accessed: 2025-06-30.

[3] LearningRO. Learningro: Ai tutor for students, 2025. Accessed: 2025-06-30.

[4] Kira Learning. Kira learning: Teaching ai in classrooms, 2025. Accessed: 2025-06-30.

[5] Riaz Ahmed Khan, M. F. Khan, M. Amin, M. N. Islam, and N. M. Nawi. A machine learning approach to student performance prediction: A case study. *Smart Learning Environments*, 8(1), 2021.

[6] Diana Córdova-Esparza et al. Ai-powered tutoring systems: A systematic review. *Information*, 16(6), 2024.

[7] Patrick Bassner et al. Evaluating iris: A socratic rag-based tutor for computer science education. *arXiv preprint arXiv:2405.08008*, 2024.

[8] Ying Liu et al. Using multimodal gpt-4v-based its for language learning. *arXiv preprint arXiv:2404.03429*, 2024.

[9] Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. Attention is all you need. *arXiv preprint arXiv:1706.03762*, 2017.

[10] Andy Yeung et al. Zero-shot llm framework for assignment grading and feedback. *arXiv preprint arXiv:2501.14305*, 2025.

[11] Emmanuel Livathinos et al. Docling: A toolkit for document conversion using lightweight transformers. *arXiv preprint arXiv:2501.17887*, 2025.

[12] Franziska Neumann et al. An llm-driven chatbot in higher education for databases and information systems. *ResearchGate Preprint*, 2025.